



单位代码 10006

学 号 22371240

分 类 号 (中图分类号)

北京航空航天大学

BEIHANG UNIVERSITY

毕业设计 (论文)

面向深度学习系统模糊测试的测试用例
智能管理与分析

学 院 名 称 计算机学院

专 业 名 称 计算机科学与技术专业

学 生 姓 名 郭俊杰

指 导 教 师 吴际

2026 年 5 月

北京航空航天大学

本科生毕业设计（论文）任务书

I、毕业设计（论文）题目：

面向深度学习系统模糊测试的测试用例智能管理与分析

II、毕业设计（论文）使用的原始资料（数据）及设计技术要求：

原始资料及设计要求第一行

原始资料及设计要求第二行

原始资料及设计要求第三行

原始资料及设计要求第四行

原始资料及设计要求第五行

III、毕业设计（论文）工作内容：

工作内容第一行

工作内容第二行

工作内容第三行

工作内容第四行

工作内容第五行

工作内容第六行

IV、主要参考资料：

参考文献第一行

参考文献第二行

参考文献第三行

参考文献第四行

参考文献第五行

参考文献第六行

参考文献第七行

参考文献第八行

____ 计算机 ____ 学院 计算机科学与技术 专业类 ____ 220611 ____ 班

学生 ____ 郭俊杰 ____

毕业设计(论文)时间： ____ (2026) 年 ____ (1) 月 ____ (1) 日至 ____ (2026) 年 ____ (6) 月 ____ (1) 日

答辩时间： ____ (2026) ____ 年 ____ (6) 月 ____ (1) 日

成 绩： ____

指导教师： ____

兼职教师或答疑教师（并指出所负责部分）：

____ 系（教研室）主任（签字）： _____

注：任务书应该附在已完成的毕业设计（论文）的首页。



本人声明

我声明，本论文及其研究工作是由本人在导师指导下独立完成的，在完成论文时所利用的一切资料均已在参考文献中列出。

作者： 郭俊杰

签字：

时间： 2026 年 5 月



面向深度学习系统模糊测试的测试用例智能管理与分析

学 生： 郭俊杰

指导教师： 吴际

摘 要

深度学习框架（如 PyTorch、TensorFlow）作为人工智能应用的核心基础设施，其底层算子实现的可靠性直接影响上层模型训练与推理的正确性。模糊测试（Fuzzing）是检测此类框架底层缺陷的有效手段，但在实际应用中面临三项关键挑战：（1）大量生成的测试用例中存在高比例的冗余样本，导致执行耗时与覆盖收益不成正比；（2）框架崩溃日志通常为非结构化的长文本堆栈，人工排查根因耗时且依赖领域经验；（3）测试用例的生成缺乏覆盖反馈指导，容易陷入局部变异的盲目循环。

针对上述问题，本文设计并实现了一套基于语义检索与大语言模型（LLM）的智能化模糊测试辅助分析平台。系统采用前后端分离架构，以 Vue 3 构建前端交互界面，以 Python FastAPI 承载算法后端，以 Java Spring Boot 提供业务数据管理，以 MySQL 与 Milvus 组成的异构混合存储底座支撑结构化元数据与高维语义向量的协同管理。

在测试效率优化方面，本文提出了覆盖效能指数（Coverage Efficiency Index, CEI）模型。CEI 综合考量每条测试用例的分支覆盖贡献与加权计算成本，通过动态百分位阈值筛选机制剔除低效用例。在 2000 例模拟实验中，以不足 4% 的覆盖率损失实现了 60% 的执行耗时缩减；在 1000 例真实物理实验中，Gcov/LCOV 探针采集的物理命中行数仅下降 0.005%，总执行耗时压减 34.5%。

在故障根因分析方面，本文提出了双空间 Daikon 边界界定引擎。该引擎召回与目标崩溃用例拓扑相同的健康组与崩溃组样本，将嵌套张量参数展平为一维数值键值对后，在多模板扫描框架下执行不变量推断。仅当某数学约束在健康组全票通过且崩溃组全票否决时，才将其标记为导致崩溃的“数学红线”。实验表明，引擎在五条已知约束上实现了 100% 的召回率，并在真实数据上自动发现了跨算子的隐式整除约束等非平凡物理红线，实现了从“现象记录”到“因果解释”的能力提升。

在智能化闭环方面，本文基于检索增强生成（Retrieval-Augmented Generation, RAG）技术，将语义检索召回的相似用例上下文与 Daikon 数学红线注入结构化四板块提示词



模板，驱动 LLM 生成具备项目环境感知的诊断报告与靶向补盲测试脚本。RAG 辅助诊断在准确性评分上较直接 LLM 输出提升 50%，幻觉程度降低 61%。

系统还实现了基于 multiprocessing.spawn 的进程级隔离执行引擎，通过捕获子进程退出状态码安全拦截底层 C++ 段错误与断言失败，保障主服务持续可用。前端基于 WebGL 渲染方案在 20000 散点规模下仍保持 57 FPS 的流畅交互体验。

关键词：深度学习框架，模糊测试，语义检索，检索增强生成，覆盖效能指数



Intelligent Management and Analysis of Test Cases for Deep Learning System Fuzzing

Author: Guo Junjie

Tutor: Wu Ji

Abstract

Deep learning frameworks such as PyTorch and TensorFlow serve as critical infrastructure for artificial intelligence applications, where the reliability of low-level operator implementations directly affects the correctness of model training and inference. Fuzz testing is an effective technique for detecting defects in such frameworks; however, its practical deployment is hindered by three key challenges: (1) a high proportion of redundant test cases that consume substantial execution time with disproportionately low coverage gains; (2) unstructured crash logs that require significant manual effort and domain expertise to diagnose root causes; and (3) a lack of coverage-guided feedback in test case generation, which leads to blind and repetitive mutations.

To address these challenges, this thesis designs and implements an intelligent fuzz testing analysis platform based on semantic retrieval and large language models (LLMs). The system adopts a decoupled front-end/back-end architecture, with Vue 3 for the interactive front end, Python FastAPI for the algorithmic back end, Java Spring Boot for business data management, and a heterogeneous storage layer composed of MySQL and Milvus for the coordinated management of structured metadata and high-dimensional semantic vectors.

For test efficiency optimization, a Coverage Efficiency Index (CEI) model is proposed. CEI jointly considers the branch coverage contribution and the weighted computational cost of each test case, and eliminates low-value cases through a dynamic percentile threshold mechanism. In a simulated experiment with 2,000 cases, less than 4% coverage loss was incurred while achieving a 60% reduction in execution time. In a physical experiment with 1,000 cases instrumented with Gcov/LCOV probes, the number of physically hit lines decreased by only 0.005%, while total execution time was reduced by 34.5%.



For fault root-cause analysis, a dual-space Daikon boundary delineation engine is proposed. The engine retrieves healthy and crashed samples that share the same operator topology as the target crash case, flattens nested tensor parameters into one-dimensional key-value pairs, and performs invariant inference under a multi-template scanning framework. A mathematical constraint is marked as a “red line” only when it is unanimously satisfied by all healthy samples and unanimously violated by all crashed samples. Experiments show that the engine achieves a 100% recall rate on five known constraints and automatically discovers non-trivial implicit divisibility constraints across operators on real-world crash data, elevating the analysis from mere phenomenon recording to causal explanation.

For intelligent closed-loop automation, Retrieval-Augmented Generation (RAG) is employed to inject retrieved similar-case contexts and Daikon-inferred red lines into a structured four-block prompt template, driving the LLM to generate project-aware diagnostic reports and targeted gap-filling test scripts. RAG-assisted diagnosis improves accuracy scores by 50% and reduces hallucination levels by 61% compared to direct LLM output.

The system also implements a process-level isolated execution engine based on `multiprocessing.spawn`, which safely intercepts low-level C++ segmentation faults and assertion failures by capturing child process exit codes, thereby ensuring the continuous availability of the main web service. The front end employs a WebGL-based rendering scheme that maintains 57 FPS interactive performance at 20,000 scatter points.

Key words: Deep Learning Framework, Fuzz Testing, Semantic Retrieval, Retrieval-Augmented Generation, Coverage Efficiency Index



目 录

1 绪论	5
1.1 课题背景	5
1.1.1 深度学习框架的工程地位与安全隐患	5
1.1.2 模糊测试的引入与测试数据管理瓶颈	5
1.2 国内外研究现状	6
1.2.1 深度学习框架模糊测试方法研究	6
1.2.2 测试数据管理与效能评估相关研究	7
1.2.3 大语言模型辅助软件测试研究现状	7
1.3 研究目标与主要工作	8
1.4 论文组织结构	8
2 相关技术与理论基础	10
2.1 深度学习框架与模糊测试技术	10
2.1.1 深度学习框架的体系结构	10
2.1.2 模糊测试技术原理与分类	10
2.2 文本语义表征与向量检索技术	11
2.2.1 基于孪生网络的预训练语义编码模型	11
2.2.2 高维向量近似最近邻检索	11
2.3 降维分析与不变量推断方法	12
2.3.1 主成分分析原理	12
2.3.2 Daikon 动态不变量检测框架	12
2.4 检索增强生成技术	13
2.5 代码覆盖率采集技术	13
2.6 本章小结	13



3 系统需求分析与总体设计	15
3.1 系统需求分析	15
3.1.1 功能性需求	15
3.1.2 非功能性需求	15
3.2 系统总体架构设计	16
3.2.1 前后端分离与混合后端架构	16
3.2.2 异构双库存储架构	17
3.2.3 系统技术栈选型说明	18
3.3 系统功能模块划分与数据流	18
4 系统详细设计与核心算法	20
4.1 自动化日志解析与双路向量化数据流水线	20
4.1.1 基于分层正则匹配的日志解析引擎	20
4.1.2 基于 DFS 拓扑回溯的故障溯源定位引擎	21
4.1.3 双路向量化特征表征策略	22
4.2 异构混合协同存储与生命周期管理	23
4.2.1 结构化元数据与向量特征的隔离协同设计	23
4.2.2 异步协程削峰与高并发双写一致性保障	24
4.2.3 基于 CEI 阈值的跨库级联清理机制	24
4.3 双空间 Daikon 故障边界定界引擎	25
4.3.1 对照组召回与参数展平机制	25
4.3.2 绝对数学规律探测与物理红线推断	26
4.3.3 大模型软解释与双轨排障流水线	27
4.4 测试集多样性度量与 CEI 效能评估模型	29
4.4.1 基于 PCA 的高维特征降维与空间分布监测	29
4.4.2 覆盖效能指数量化模型	30
4.4.3 算子组合缺口的关联分析机制	31
4.5 基于 RAG 的智能化闭环机制	31
4.5.1 环境感知辅助诊断	31



4.5.2 算子缺口靶向补盲与用例自动生成	32
4.6 本章小结	33
5 系统实现与功能展示	34
5.1 开发环境与工具配置	34
5.2 后端核心模块实现	35
5.2.1 Python FastAPI 算法后端实现	35
5.2.2 Java Spring Boot 业务后端实现	37
5.2.3 数据库表结构与向量集合详细设计	37
5.3 前端界面实现与功能展示	39
5.3.1 系统总览仪表盘	39
5.3.2 用例管理与智能语义检索界面	40
5.3.3 故障边界对比与 Daikon 定界展示界面	41
5.3.4 算子覆盖热力图与 AI 补盲界面	42
5.3.5 CEI 效能分析与智能清退界面	43
5.3.6 PCA 多样性聚类监控界面	45
5.4 底层物理执行沙箱实现	46
5.4.1 Gcov/LCOV 探针的编译挂载流程	47
5.4.2 基于 multiprocessing.spawn 的隔离执行引擎	48
6 实验与结果分析	51
6.1 实验环境与数据集	51
6.2 CEI 筛选算法的有效性验证	51
6.2.1 模拟验证实验	52
6.2.2 真实物理对照实验 (200 例)	53
6.2.3 千例规模物理跑测论证	54
6.3 Daikon 边界定界引擎的有效性验证	55
6.3.1 已知数学规律的精确捕获实验	55
6.3.2 未知边界条件的探索性发现	56



6.4 语义检索与 RAG 辅助诊断效果评估	57
6.5 系统性能与可扩展性分析	58
6.6 本章小结	59
总结与展望	61
致谢	63
参考文献	64



1 绪论

1.1 课题背景

1.1.1 深度学习框架的工程地位与安全隐患

深度学习(Deep Learning, DL)框架在当前人工智能技术栈中承担底层基础设施的角色。以 PyTorch 和 TensorFlow 为代表的主流框架,向下屏蔽异构计算硬件的物理特性与内存分配机制,向上提供高度封装的数学算子(Operator)库与自动微分接口。这种分层封装降低了大规模神经网络的开发门槛。然而,框架自身已演进为由数百万行代码构成的超大型系统软件,其内部潜伏的逻辑缺陷也在持续积累 [1]。

与传统应用软件直接抛出异常中断不同,DL 框架的缺陷往往具有隐蔽性。在处理特定边界条件的张量维度组合或执行混合精度计算时,底层算子可能在正向传播或梯度计算过程中产生微小的数值截断误差。这种偏差在运行时通常不会触发系统级警告。上层神经网络在包含逻辑错误的框架上静默完成训练后,会将偏差以权重的形式吸收为模型自身的隐性缺陷 [2]。一旦此类缺陷被部署至安全关键场景——如自动驾驶的环境感知模块或医疗影像的病灶识别系统——将引发系统漏检与误判。此外,框架底层的内存越界或缓冲区溢出漏洞也存在被攻击者利用的风险。攻击者可通过构造包含畸形张量的计算图(Computation Graph)触发非法内存访问,进而威胁云端算力集群的数据安全 [3]。

1.1.2 模糊测试的引入与测试数据管理瓶颈

针对上述框架深层逻辑异常,传统基于人工编写固定输入的单元测试方法难以覆盖框架庞大的输入空间。该空间涵盖多变的数据类型、任意的张量维度以及数百种算子的嵌套拓扑组合。因此,学术界与工业界广泛引入了模糊测试(Fuzzing)技术 [4]。该技术通过自动化算法高频生成包含畸形参数或极端边界状态的测试输入,持续送入待测框架执行,以主动探索深层代码段中的潜在漏洞。

然而,高度自动化的变异引擎在连续运行期间会产出规模庞大的非结构化测试产物。这些产物包含两部分:一是测试用例本身,其形态为包含随机生成的高维算子组合与属性配置的代码脚本;二是运行时产生的报错日志,此类日志交织着应用层调用栈回



溯、底层 C++ 执行引擎的异常断点以及显卡架构的内存转储信息。在分布式算力支持下,全天候运行的测试工具每日可产生数十万个独立脚本及数十吉字节的日志文本。这种数据体量的膨胀与结构的无序,使得后续漏洞溯源与效能评估面临以下三方面核心痛点。

其一,缺陷回溯与根因定位存在检索困境。系统每日捕获的数百次崩溃警告中,表层大量报错往往由底层少数几个相同漏洞引起。传统基于关键字匹配的文本检索工具在面对包含跨语言调用、变量名随机变异的崩溃日志时,特征匹配精度受到严重制约。同时,底层计算引擎对指令调度的高度封装,使得异常堆栈难以自动映射至应用层的具体触发算子,拉长了从根因定位到缺陷修复的周期 [5]。

其二,测试用例多样性缺乏有效的评估手段。模糊测试的有效性依赖于生成算法对输入空间的探索广度。变异引擎在实际运行中容易陷入局部最优状态,反复生成拓扑结构雷同的代码逻辑。针对包含节点属性与复杂边连接关系的高维计算图数据,传统关系型数据库与基础统计图表难以对其分布特征进行有效表征与度量 [6]。

其三,测试效能分析的维度过于单一。现阶段对框架模糊测试的评估常局限于“代码覆盖率”或“用例是否异常”等粗粒度标量指标。这种模式在入库时剥离了计算图自身的深层语义特征,且忽略了单个用例执行所消耗的时间与显存成本。系统难以量化评估单体用例的投入产出比,也无法自动剥离低价值冗余用例 [7]。此外,现有 workflow 缺乏从历史失效测试集中提取算子盲区特征并将其转化为定向测试资产的闭环能力。

1.2 国内外研究现状

1.2.1 深度学习框架模糊测试方法研究

面向 DL 框架的模糊测试是近年来软件测试领域的研究热点。Pei 等人提出的 DeepXplore[8] 引入了神经元覆盖率的概念,通过差分测试策略自动生成能够触发系统内部逻辑差异的测试输入,为后续研究提供了理论基础。Xie 等人 [9] 在此基础上提出了 DeepHunter,将变异策略从像素级扰动扩展至覆盖率引导的多层次变异机制。

在框架级别的测试方面,Wei 等人提出的 FreeFuzz[10] 从开源代码中自动提取 API 调用模式来构造测试用例,以较低成本覆盖了 PyTorch 与 TensorFlow 的大量框架接口。Deng 等人 [11] 进一步探索了利用大语言模型 (Large Language Model, LLM) 生成框架测试用例的可行性,验证了 LLM 在理解框架 API 语义方面的潜力。上述工作在用例自动化生成方面取得了重要进展,但研究重心集中于变异策略与生成算法的优化,对测试



产物在生成之后的管理、归约与效能分析关注不足。当生成端的产出规模持续膨胀时,管理端的滞后成为制约整体测试效能的瓶颈。

1.2.2 测试数据管理与效能评估相关研究

在测试用例管理领域,传统研究主要聚焦于用例优先级排序与用例约简两个方向。Rothermel 等人 [12] 提出了基于覆盖率的贪心排序策略,通过优先执行覆盖新代码分支最多的用例来提升回归测试效率。Harrold 等人 [13] 研究了在维持覆盖率约束下的最小用例集选择问题。这些方法在传统软件测试中已较为成熟。但面对 DL 框架的测试产物时,其局限性较为明显:传统方法处理的输入通常为标量参数或简单数据结构,而 DL 测试用例的核心特征是高维计算图拓扑与多维嵌套的张量配置。基于文本匹配或覆盖率向量的相似度计算方式,难以捕捉此类数据的深层语义关联。

近年来,向量数据库(Vector Database)技术的发展为高维非结构化数据的管理开辟了新的路径 [14]。以 Milvus 为代表的向量检索引擎支持对稠密浮点向量执行近似最近邻(Approximate Nearest Neighbor, ANN)查询,能够在亚秒级响应时间内完成百万级向量的相似度匹配。将此类技术引入测试数据管理,有望解决传统关键字检索在处理非结构化崩溃日志时精度不足的问题。然而,将向量检索与结构化元数据存储协同应用于模糊测试数据管理的研究仍较为有限。

1.2.3 大语言模型辅助软件测试研究现状

LLM 在软件工程领域的应用正受到广泛关注 [15]。在代码生成方面,Codex[16] 与 CodeLlama[17] 等模型展示了根据自然语言描述自动生成程序代码的能力。在缺陷修复方面,多项研究表明 LLM 能够基于错误上下文生成候选修复补丁 [18]。这些工作为模糊测试中的用例生成与故障诊断提供了新的技术手段。

然而,通用 LLM 在处理垂直测试领域的任务时存在明确的局限。脱离具体测试环境与历史运行数据,模型在面对高度封装且语法专业的异常堆栈时,容易产生“幻觉”(Hallucination)现象 [19]。其具体表现为虚构底层调度逻辑或推荐已废弃的参数配置。为缓解这一问题,检索增强生成(Retrieval-Augmented Generation, RAG)架构 [20] 通过在推理前从外部知识库中召回相关上下文来约束模型的输出边界,在多个垂直领域中已取得积极效果。但将 RAG 系统性地应用于 DL 框架的故障诊断与测试用例定向生成,相关研究仍处于起步阶段。



1.3 研究目标与主要工作

本课题针对 DL 框架模糊测试产物在管理与分析环节的薄弱现状，旨在设计并实现一套面向异构非结构化测试数据的智能管理与效能分析系统。具体研究目标涵盖以下三个方面。

第一，构建跨模态关联映射的异构存储架构。针对测试过程中产生的计算图脚本、异常崩溃日志及覆盖率元数据，设计结构化元数据与高维语义向量的协同存储方案。该方案旨在解决传统关系型数据库在处理高维语义数据时的性能瓶颈，支撑具备语义感知能力的缺陷用例检索与归类。

第二，建立用例多样性监测与效能量化评估机制。利用降维映射算法将高维计算图特征投射至低维平面，实现对测试集空间探索广度的可视化监控。同时，构建覆盖效能指数（Coverage Efficiency Index, CEI）评估模型，量化单体用例对代码覆盖的贡献度与计算成本之比，以此精准识别并过滤低价值冗余用例。

第三，实现基于语义增强的故障辅助诊断与闭环补盲。整合 RAG 技术与 LLM 的推理能力，构建具备项目环境感知的辅助诊断助手。此外，通过动态识别算子覆盖缺口驱动 LLM 定向生成测试脚本，形成“特征分析—覆盖度评估—定向生成”的智能化闭环反馈流程。

本文的主要工作包括：设计并实现自动化日志解析与双路向量化数据流水线；部署 MySQL 与 Milvus 协同的异构混合存储底座；开发基于深度优先搜索（Depth-First Search, DFS）拓扑回溯的故障溯源定位引擎；提出并实现双空间 Daikon 故障边界界定算法；构建基于主成分分析（Principal Component Analysis, PCA）的测试集多样性降维监测机制与 CEI 效能量化模型；集成基于 RAG 架构的智能诊断助手与算子缺口靶向补盲模块；在插桩编译的 PyTorch 环境中完成千例规模的物理对照实验，验证了系统的工程有效性。

1.4 论文组织结构

本文共分为六章，各章内容安排如下。

第一章为绪论。本章阐述课题的研究背景与工程痛点，梳理国内外相关研究现状，明确研究目标与主要工作内容。

第二章介绍相关技术与理论基础。本章对 DL 框架结构、模糊测试技术、语义编码与向量检索方法、降维分析与不变量推断理论、RAG 架构以及代码覆盖率采集技术进



行阐述，为后续章节提供理论支撑。

第三章进行系统需求分析与总体架构设计。本章明确系统的功能性与非功能性需求，阐述前后端分离的混合后端架构与异构双库存储方案的设计决策，给出系统功能模块的整体划分。

第四章详细描述系统的核心算法与模块设计。本章涵盖日志解析引擎、DFS 故障溯源定位引擎、双空间 Daikon 边界定界引擎、CEI 效能评估模型、RAG 诊断助手与闭环补盲机制的算法细节。

第五章展示系统的工程实现与功能界面。本章介绍后端服务的关键接口实现、数据库表结构设计、各功能页面的交互逻辑以及底层物理执行沙箱的构建过程。

第六章设计并执行验证实验。本章通过模拟实验与真实物理对照实验评估 CEI 筛选算法的有效性，验证 Daikon 边界定界引擎的准确性，并对系统整体性能指标进行分析。

最后为总结与展望，回顾本课题的主要成果，分析现有系统的不足并讨论后续改进方向。



2 相关技术与理论基础

2.1 深度学习框架与模糊测试技术

2.1.1 深度学习框架的体系结构

当前主流 DL 框架普遍采用分层架构设计 [21]。以 PyTorch 为例,其体系结构自顶向下可划分为三个层次。最上层是面向用户的 Python 前端接口,提供张量操作、自动微分以及神经网络模块的高级抽象。中间层为计算图的中间表示 (Intermediate Representation),负责将用户定义的运算逻辑转化为有向无环图 (Directed Acyclic Graph, DAG) 结构,其中节点对应算子调用,有向边描述数据的流向依赖关系。最底层是 C++ 实现的 ATen 张量计算库与 CUDA 核函数调度引擎,直接操控 GPU 显存分配、并行线程调度与底层数值运算。

这种三层结构的设计使得框架接口简洁、执行效率高。但同时也意味着,当底层 ATen 库中某个算子存在边界条件处理缺陷时,错误会被上层抽象层层封装,上层用户极难通过 Python 级别的堆栈信息追溯至真实的缺陷触发位置。本课题所构建的故障溯源引擎正是针对这一跨层封装带来的定位困难而设计。

2.1.2 模糊测试技术原理与分类

Fuzzing 的核心思想是通过自动化手段持续生成大量测试输入,将其送入待测程序执行,并监控运行时的异常信号 [22]。依据输入的构造策略, Fuzzing 可分为两大类别:基于变异 (Mutation-based) 的方法从已有的种子输入出发,通过随机翻转、插入或删除字节来派生新的测试用例 [23];基于生成 (Generation-based) 的方法则依据输入格式的语法规则或语义模型从零构建符合特定约束的测试输入 [24]。

面向 DL 框架的 Fuzzing 面临两项特有挑战。其一,输入空间维度远高于传统软件:测试用例不再是单一的二进制文件,而是包含多维张量属性、算子类型枚举及其拓扑组合的复合结构。其二,“崩溃”的定义更为复杂:除段错误与未捕获异常外,数值精度的静默偏离同样构成框架缺陷,但此类异常不会触发操作系统信号,需要额外的差分预言机制进行检测 [25]。本课题的测试管理系统需要对上述异构崩溃产物统一归档并进行特征提取,这对底层数据流水线的泛化能力提出了明确要求。



2.2 文本语义表征与向量检索技术

2.2.1 基于孪生网络的预训练语义编码模型

传统的词袋模型或 TF-IDF 方法将文本映射为稀疏高维向量, 无法捕捉词序与上下文语义信息 [26]。Reimers 等人提出的 Sentence-BERT[27] 在预训练语言模型的基础上引入孪生网络 (Siamese Network) 结构, 通过对比学习目标使模型输出的句子级嵌入向量在语义空间中具备距离可度量性。具体而言, 两条输入文本分别经过共享参数的编码器后, 各自产生一个固定维度的稠密浮点向量, 余弦相似度即可直接反映两段文本的语义接近程度。

本课题选用 all-MiniLM-L6-v2[28] 作为具体的编码器实例。该模型基于 6 层 Transformer 架构蒸馏而来, 输出维度为 384 维, 在保持较高语义区分度的同时将推理延迟控制在毫秒级。在本系统中, 该模型被部署于数据摄入流水线的向量化环节, 负责将崩溃日志的报错堆栈文本编码为稠密语义向量, 为后续的相似缺陷检索与聚类分析提供数值基础。

2.2.2 高维向量近似最近邻检索

当向量集合的规模增长至百万级别时, 逐一计算查询向量与所有库中向量之间的距离将带来不可接受的延迟。ANN 检索通过牺牲少量精度来换取数量级的速度提升 [29]。Milvus[30] 是一款面向海量向量数据的开源检索引擎, 支持多种索引类型与距离度量方式。

本课题选用 IVF_FLAT (Inverted File Index with Flat quantization) 作为主索引结构。该索引首先通过聚类算法将向量空间划分为若干 Voronoi 单元, 查询时仅在距离查询点最近的若干单元内执行精确扫描, 从而将搜索范围从全量集合缩减至局部子集 [31]。

向量间的距离度量采用余弦相似度。对于两个 d 维向量 $\mathbf{a}$ 与 $\mathbf{b}$, 其余弦相似度定义为:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \cdot \sqrt{\sum_{i=1}^d b_i^2}} \quad (2.1)$$

其中 a_i 与 b_i 分别为向量 $\mathbf{a}$ 和 $\mathbf{b}$ 的第 i 个分量。该度量对向量的模长不敏感, 仅关注方向一致性, 适用于本系统中长度不等的崩溃文本经编码后产生的语义向量之间的相似度计算。



2.3 降维分析与不变量推断方法

2.3.1 主成分分析原理

PCA 是一种经典的线性降维方法 [32], 其目标是将高维数据投射至低维子空间的同时保留最大方差信息。给定 n 个 d 维样本构成的数据矩阵 $\mathbf{X} \in \mathbb{R}^{n \times d}$ (已中心化), PCA 首先计算样本协方差矩阵:

$$\mathbf{C} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \quad (2.2)$$

其中 $\mathbf{C} \in \mathbb{R}^{d \times d}$ 为对称半正定矩阵。随后对 $\mathbf{C}$ 执行特征值分解:

$$\mathbf{C} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad k = 1, 2, \dots, d \quad (2.3)$$

其中 λ_k 为第 k 个特征值, $\mathbf{v}_k$ 为对应的单位特征向量。将特征值按降序排列后, 取前 p 个特征向量构成投影矩阵 $\mathbf{W} = [\mathbf{v}_1, \dots, \mathbf{v}_p] \in \mathbb{R}^{d \times p}$ 。原始数据通过 $\mathbf{Y} = \mathbf{XW}$ 即可降阶至 p 维表示。

本课题将 384 维的测试用例语义向量通过 PCA 降至 2 维平面坐标, 用于生成测试集多样性的空间分布散点图。该降维结果为研究人员提供了对测试集探索广度的直观监测手段。

2.3.2 Daikon 动态不变量检测框架

Daikon[33] 是由 Ernst 等人提出的动态不变量检测工具。其核心思想是: 从程序多次执行的运行时轨迹中, 自动推断可能成立的程序不变量 (Program Invariant)。具体而言, Daikon 在被观测程序的关键执行点采集变量的实际取值, 然后将这些取值序列与预定义的不变量模板库 (如 $x > 0$ 、 $x = a \cdot y + b$ 、 $x \in \{v_1, v_2, \dots\}$ 等) 进行匹配。若某一模板在全部已观测轨迹上均成立, 则将其报告为一条候选不变量。

本课题借鉴了 Daikon 的核心推断范式, 但应用场景与经典用法存在本质差异。经典 Daikon 面向的是单一程序的正常执行轨迹, 而本系统面对的是 DL 框架 Fuzzing 产物中”成功执行”与”崩溃触发”两组对照样本的参数空间。系统从中推断出区分两组样本的数学边界条件, 将其作为算子输入参数的物理约束红线。这一改造的具体算法设计将在第四章详细阐述。



2.4 检索增强生成技术

RAG 架构 [34] 由检索器 (Retriever) 与生成器 (Generator) 两个核心组件构成。当用户发起查询时, 检索器首先从外部知识库中召回与查询语义相关的文档片段, 随后将这些片段作为上下文拼接至 LLM 的输入提示词中, 引导生成器在受约束的知识范围内产出回答。

这一架构相较于直接调用 LLM 具有两方面优势。其一, 通过外部知识注入减轻模型参数记忆负担, 降低 Hallucination 的发生概率。其二, 知识库可动态更新而无需重新训练模型权重, 使系统能够适应持续增长的领域数据 [35]。

本课题在两个场景中集成 RAG 机制: 一是故障辅助诊断——以当前崩溃用例的语义向量召回历史相似案例, 为 LLM 提供具体的项目环境上下文; 二是算子缺口补盲——将识别出的未覆盖算子组合信息注入提示词, 驱动 LLM 生成具有针对性的测试脚本。RAG 在本系统中的具体集成方式将在第四章展开。

2.5 代码覆盖率采集技术

代码覆盖率是衡量测试完备程度的基础指标 [36]。GNU 工具链中的 Gcov[37] 是面向 C/C++ 程序的覆盖率采集工具。其工作原理分为三个阶段: 在编译阶段, 通过向 GCC 传递 `-coverage` 选项, 编译器在源代码的每个基本块入口处插桩 (Instrumentation) 计数器指令, 同时生成记录源码结构信息的 `.gcno` 文件; 在执行阶段, 程序运行时各计数器累加实际命中次数, 进程正常退出后将计数结果落盘为 `.gcda` 文件; 在分析阶段, Gcov 读取 `.gcno` 与 `.gcda` 文件, 计算并输出每个源文件的行级与分支级覆盖率。

LCOV[38] 是对 Gcov 输出的上层封装工具。它支持对多次运行产生的覆盖率数据执行合并 (Merge) 与差集运算, 并可生成 HTML 格式的可视化覆盖率报告。在本课题中, 系统需要在插桩编译的 PyTorch 底层 C++ 引擎上逐条执行测试用例并采集对应的覆盖率增量数据。LCOV 的合并功能使系统能够追踪每条用例对全局覆盖率的边际贡献, 这一数据是计算 CEI 的必要输入。

2.6 本章小结

本章介绍了本课题涉及的核心技术与理论基础。首先阐述了 DL 框架的分层体系结构与 Fuzzing 技术的基本原理, 明确了待测对象与测试方法的技术特征。随后介绍了 Sentence-BERT 语义编码模型与 Milvus 向量检索引擎的工作机制, 为系统的语义检



索能力提供了技术支撑。在此基础上，说明了 PCA 降维方法与 Daikon 不变量推断的核心思想，为后续的多多样性监测与边界定界奠定了理论依据。最后概述了 RAG 架构与 Gcov/LCOV 覆盖率采集工具链，分别对应系统的智能诊断模块与效能评估模块的底层技术需求。上述技术将在第四章中结合系统的具体算法设计进行深入阐释。



3 系统需求分析与总体设计

3.1 系统需求分析

3.1.1 功能性需求

根据第一章对 DL 框架 Fuzzing 产物管理痛点的分析，本系统需要满足以下六项核心功能性需求。

(1) 自动化日志解析与特征提取。系统应能接收 Fuzzing 工具产生的原始崩溃日志与测试脚本，自动从中提取结构化元数据字段（如错误类型、触发算子、张量维度配置等），并将非结构化文本编码为稠密语义向量，以供后续检索与分析使用。

(2) 语义检索与故障溯源。系统应支持用户以自然语言描述或崩溃特征片段为查询条件，在向量空间中召回语义相近的历史用例。同时，系统应提供基于计算图拓扑关系的故障溯源能力，辅助用户定位底层算子级别的缺陷根因。

(3) 测试集多样性监测。系统应将全量测试用例的高维语义特征降维至二维平面并进行可视化展示，供研究人员监控当前测试集在语义空间中的分布密度与覆盖广度。

(4) 效能评估与低价值用例清退。系统应建立覆盖率与成本相结合的效能量化指标，识别对全局覆盖率贡献不足但执行成本偏高的冗余用例，并支持按阈值自动执行跨数据库的级联清理操作。

(5) 故障边界界定。系统应能基于成功与崩溃两组对照样本的参数分布，自动推断区分正常执行与异常触发的数学边界条件，以可视化方式呈现算子输入参数的约束红线。

(6) 智能诊断与算子缺口补盲。系统应集成 RAG 机制，在崩溃诊断场景中为 LLM 注入历史相似案例的上下文信息。同时，系统应能识别当前测试集尚未覆盖的算子组合缺口，驱动 LLM 定向生成补充测试脚本并纳入管理。

3.1.2 非功能性需求

(1) 高并发写入性能。在 Fuzzing 工具持续产生的工作场景下，系统的数据摄入接口应支持每秒数十条用例的并发写入，结构化元数据与语义向量的双库落盘过程不应出现阻塞或数据丢失。



(2) 大规模数据可视化性能。多样性监测页面需在浏览器端渲染万级规模的二维散点图。系统应在主流硬件配置下维持流畅的交互帧率。

(3) 底层执行隔离安全性。系统在执行测试用例以采集覆盖率数据时, 需要将每条用例置于独立进程中运行, 防止段错误或线程死锁等异常传播至主服务进程。

(4) 异构数据一致性。结构化元数据与语义向量分别存储于两个异构数据库中。在执行写入或删除操作时, 系统应保证两库之间的数据一致性, 避免出现单库残留的脏数据。

3.2 系统总体架构设计

3.2.1 前后端分离与混合后端架构

本系统采用前后端分离的整体架构, 后端进一步拆分为业务后端与算法后端两个独立服务。这一设计决策基于以下工程考量。

DL 领域的核心算法生态——包括语义编码模型的推理、向量数据库的客户端驱动以及科学计算库——均以 Python 作为第一语言支持。将向量化、降维分析、Daikon 边界推断等计算密集型任务封装为独立的 Python 服务, 可直接调用 PyTorch、scikit-learn 与 Milvus 的 Python SDK, 避免跨语言绑定带来的兼容性成本。该服务基于 FastAPI 框架构建, 通过异步协程 (asyncio) 机制处理并发的应用程序编程接口 (Application Programming Interface, API) 请求。

与此同时, 面向前端的业务逻辑——如用户鉴权、分页查询、关系型数据的 CRUD 操作——对运行时类型安全与成熟的对象关系映射 (Object-Relational Mapping, ORM) 工具链有明确需求。Java 生态在这一领域具有长期积累: Spring Boot 框架提供了完备的依赖注入与事务管理机制, MyBatis 支持灵活的 SQL 映射与动态查询构建。因此, 业务后端采用 Java Spring Boot 实现, 负责接收前端请求并路由至自身或算法后端。

两个后端服务通过 HTTP API 进行通信。前端所有请求统一发送至 Java 业务后端的 8081 端口, 涉及算法计算的请求由业务后端通过内部代理转发至 Python 算法后端的 8000 端口。这种代理转发模式对前端透明, 降低了前端的集成复杂度。系统总体架构如图3.1所示。

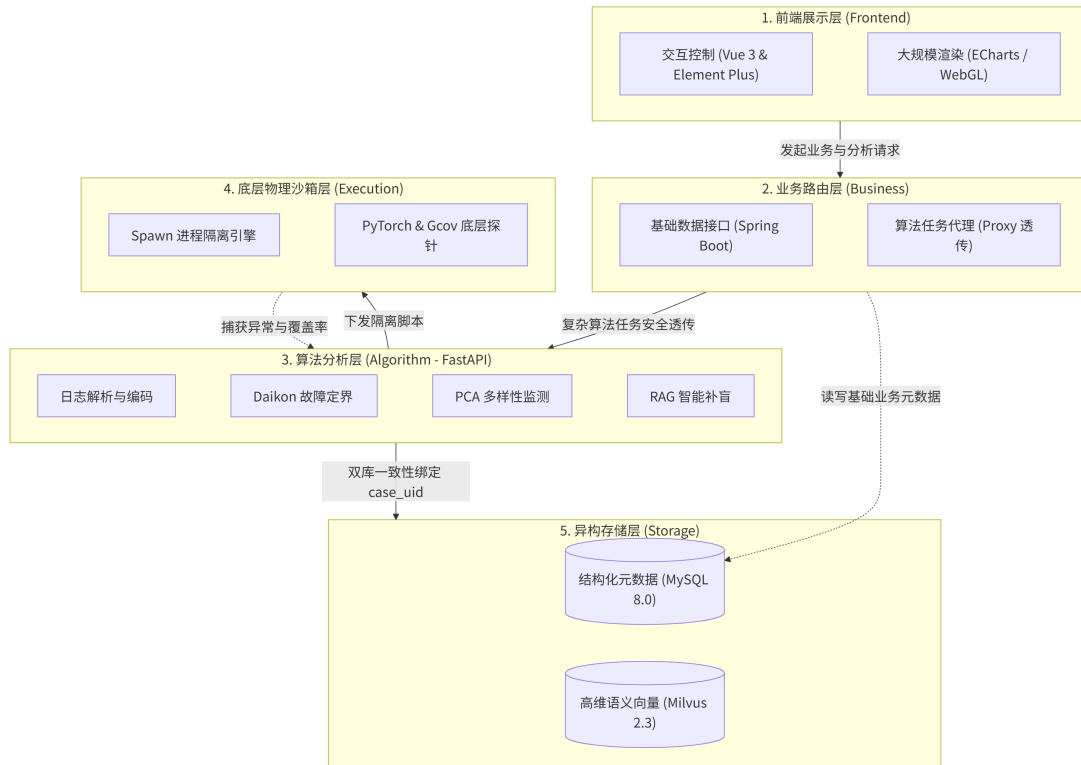


图 3.1 系统总体架构图

3.2.2 异构双库存储架构

本系统的数据资产包含两种截然不同的形态：一是结构化的元数据字段（如错误类型枚举、算子名称、张量维度、时间戳、CEI 得分等），二是由 Sentence-BERT 编码产生的 384 维稠密浮点语义向量。对这两类数据采用单一存储引擎进行统一管理，将不可避免地查询性能或功能适配性上产生瓶颈。

关系型数据库 MySQL 适用于结构化元数据的持久化存储。其成熟的索引机制（B+ 树索引）能够支撑基于枚举字段、时间范围或数值区间的条件查询与排序操作。但 MySQL 不具备对高维浮点向量执行 ANN 检索的能力，强行将 384 维向量序列化为 BLOB 字段存储将完全丧失向量空间中的距离语义。

Milvus 作为专用的向量检索引擎，支持在 IVF_FLAT 索引上对百万级 384 维向量执行亚秒级余弦相似度查询。但 Milvus 的标量过滤能力有限，不适合承载复杂的多条件组合查询与事务性写入。

基于上述分析，本系统采用 MySQL 与 Milvus 协同的异构双库存储架构。每条测试用例在入库时被分配一个全局唯一标识符（case_uid），该标识符同时写入 MySQL 的主



键列与 Milvus 向量记录的标量附属字段，作为两库之间的关联纽带。查询时，系统根据请求类型选择主查询库：结构化条件筛选由 MySQL 处理，语义相似度检索由 Milvus 处理，跨库结果通过 case_uid 执行关联拼接后返回前端。

3.2.3 系统技术栈选型说明

基于上述架构决策，本系统各层技术栈选型如下。

前端层采用 Vue 3 框架配合 TypeScript 进行开发。Vue 3 的组合式 API (Composition API) 适用于构建包含复杂状态逻辑的单页面应用。UI 组件库选用 Element Plus，其表格、表单与对话框组件可覆盖管理系统的基础交互需求。大规模散点图的渲染依赖 WebGL 加速方案，以满足万级数据点的实时交互要求。

业务后端层采用 Java 17 与 Spring Boot 3 框架。数据访问层使用 MyBatis 作为 ORM 工具，通过 XML 映射文件实现动态 SQL 的灵活拼装，便于适配多条件组合查询场景。数据库连接池采用 HikariCP，保障高并发下的连接复用效率。

算法后端层采用 Python 3.11 与 FastAPI 框架。FastAPI 原生支持异步请求处理与自动化 API 文档生成，适用于包装计算密集型算法为 RESTful 服务。该层依赖的核心 Python 库包括：sentence-transformers (语义编码)、pymilvus (Milvus 客户端)、scikit-learn (PCA 降维与统计计算) 以及 httpx (异步 HTTP 客户端)。

存储层中，MySQL 8.0 负责结构化元数据管理，Milvus 2.3.x 负责高维语义向量的索引与检索。两者均部署于本地服务器，通过各自的 SDK 与后端服务通信。

3.3 系统功能模块划分与数据流

本系统的功能模块按照职责划分为六个子系统：数据摄入与解析模块、语义检索与故障溯源模块、多样性监测模块、效能评估与清退模块、边界定界模块以及智能诊断与补盲模块。各模块均以独立的 API 端点形式封装于算法后端中，由业务后端统一调度并向前端暴露服务。

系统的端到端数据流转路径如下。原始 Fuzzing 产物（崩溃日志与测试脚本）经数据摄入接口进入系统后，首先由日志解析模块提取结构化元数据并过滤冗余调用帧。解析结果随后进入双路向量化环节，分别生成报错堆栈语义向量与图拓扑序列向量。结构化元数据落盘至 MySQL，语义向量落盘至 Milvus，两库通过 case_uid 保持关联。

入库完成后，下游分析模块可按需启动：语义检索模块接受自然语言查询，在 Milvus

中召回相似用例并从 MySQL 补全元数据后返回；多样性监测模块定期拉取全量向量并执行 PCA 降维，输出二维坐标供前端散点图渲染；效能评估模块读取覆盖率增量数据与算子权重信息，计算各用例的 CEI 得分，并根据动态阈值标记低价值用例；边界定界模块接收目标崩溃用例，召回对照样本组后输出参数边界条件；智能诊断模块整合检索结果构建 RAG 上下文，调用 LLM 生成诊断报告或补盲测试脚本。

数据在各模块间的流转路径如图3.2所示。

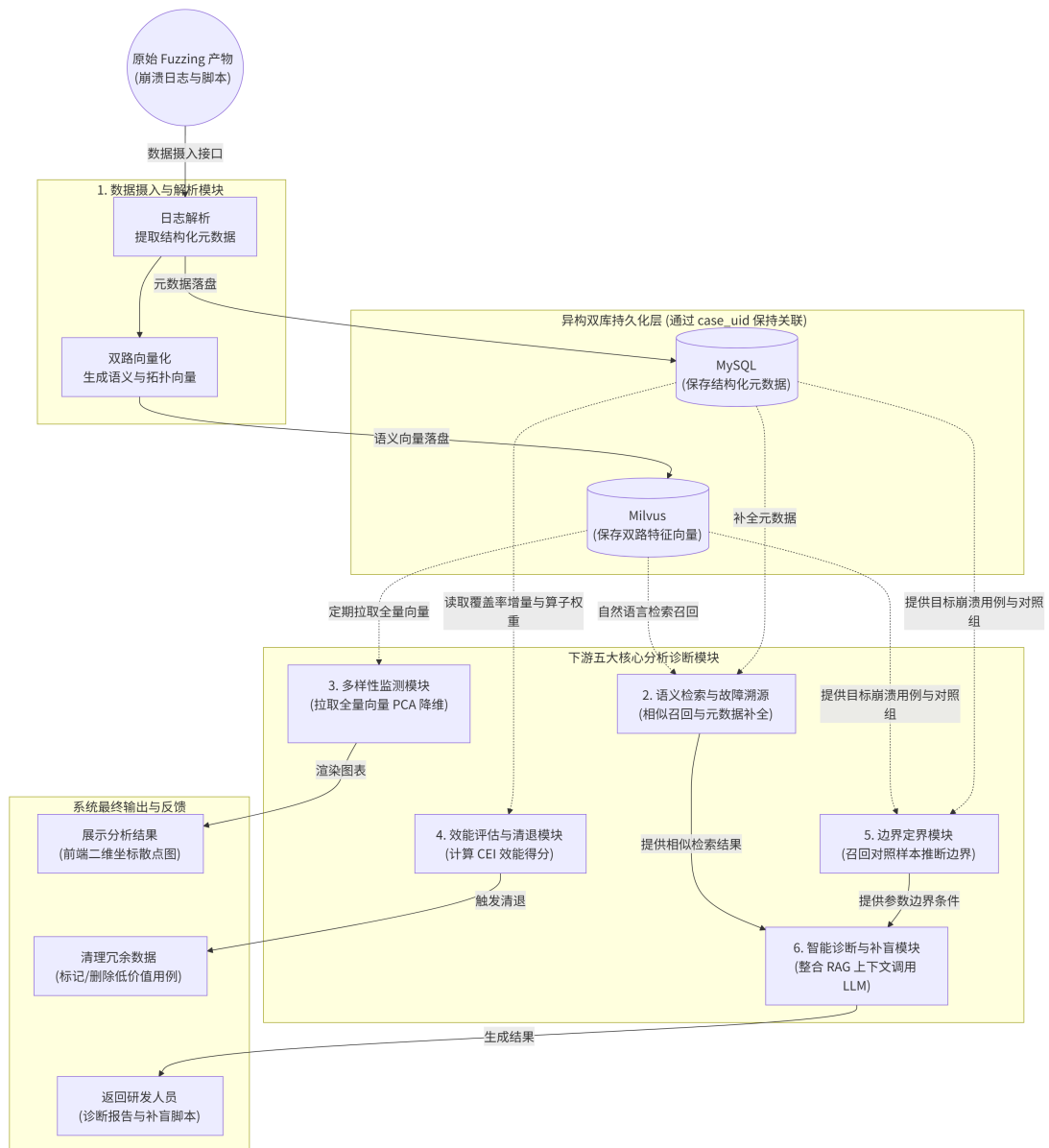


图 3.2 系统端到端数据流转路径图



4 系统详细设计与核心算法

4.1 自动化日志解析与双路向量化数据流水线

4.1.1 基于分层正则匹配的日志解析引擎

Fuzzing 工具产出的原始崩溃日志在格式上具有高度异构性。同一条崩溃记录可能混杂 Python 层异常堆栈、C++ 底层段错误信号、CUDA 运行时显存报错以及多线程交织产生的乱序输出。将此类非结构化文本转化为可供后续模块消费的标准化元数据，是整条数据流水线的入口环节。

本系统设计的 LogParser 引擎采用分层正则匹配策略，将解析过程分解为错误分类与细粒度字段提取两个独立阶段。在错误分类阶段，引擎维护一张优先级有序的错误模式表，涵盖八种面向 DL 框架的标准化错误类别，如表4.1所示。

表 4.1 LogParser 引擎支持的八种标准化错误类别

类别标识	中文描述	典型触发关键词
cuda_oom	CUDA 显存溢出	cuda out of memory, oom
shape_mismatch	张量维度不匹配	size mismatch, incompatible
segfault	内存段错误	segmentation fault, signal 11
timeout	执行超时	timeout, exceeded maximum
device_mismatch	设备不匹配	expected.*device
index_error	索引越界	index out of, out of bounds
type_error	类型错误	type error, invalid type
nan_inf	数值异常	nan, inf, not a number

引擎将输入文本统一转换为小写后，依照表中自上而下的优先级逐条扫描关键词。一旦命中即终止扫描并返回对应类别标识。优先级排列遵循“危害性递减”原则：显存溢出与段错误属于系统级致命异常，排在最前；数值异常属于静默型偏差，排在最后。对于不命中任何模式的日志，引擎将其归入 other 兜底类别。

在字段提取阶段，引擎针对每种数据类型设计了专用正则表达式族，具体提取的结构化字段包括以下六项：

(1) 错误类型 (error_type)。通过匹配 Python 异常类名模式（如 `(\w+Error):`）提



取。对于 C++ 层的段错误，引擎匹配 Segmentation Fault 字样并映射为统一标识 SegmentationFault。

(2) 张量维度 (tensor_shapes)。DL 框架的维度报错存在三种文本格式。PyTorch 使用方阵表示法 m1: [32 x 512]; TensorFlow 使用函数调用表示法 size(32, 512); 部分报错还包含匿名格式 [32 x 512]。引擎为三种格式分别编写正则表达式并按优先级依次尝试，成功匹配后输出为 {name, shape} 结构化字典列表。

(3) 框架源码位置 (source_location)。C++ 底层报错包含形如 at /pytorch/.../THTensorMath.cpp:41 的路径信息，Python 层报错遵循 File "xxx.py", line 42 格式。引擎分别以两组正则提取文件名、行号与模块路径。

(4) 显存信息 (memory_info)。仅在 cuda_oom 类别下激活。引擎提取三组数值：请求分配量、已分配量与设备总容量。

(5) 设备信息 (device_info)。通过全局扫描 cuda:N 与 cpu 标识符提取，去重后输出为列表。

(6) 原始消息 (raw_message)。保留未经处理的完整日志文本，供 RAG 模块作为 LLM 上下文素材。

面对多线程交织的脏日志，引擎在解析完整执行日志时采用逐行状态机策略。每行独立匹配时间戳模式 [YYYY-MM-DD HH:MM:SS] 与日志级别标签 [INFO|ERROR|CRASH|SUCCESS|TIMEOUT]。当检测到 CRASH 级别时，引擎提取该行中的执行耗时数值并将最终状态标记为崩溃；当检测到 ERROR 级别时，引擎暂存该行作为后续结构化解析的输入。不含有效级别标签的行视为前一条日志的续行或冗余调用帧，直接过滤。整条日志的最终输出包含事件时间线列表、最终状态枚举、总执行耗时与末尾错误的结构化解析结果。

4.1.2 基于 DFS 拓扑回溯的故障溯源定位引擎

日志解析完成后，系统需要将结构化错误信息映射回测试用例的计算图结构，定位出最可能触发崩溃的“嫌疑算子”。本系统设计的 FaultLocator 引擎基于逆向邻接矩阵与 DFS 回溯机制实现该映射。

逆向邻接矩阵的构建过程如下。每个测试用例包含一条算子连接链，以有向箭头分隔的字符串形式表示，例如 Conv2d->BatchNorm2d->ReLU->Linear。引擎首先对该字符串进行分词，以-> 为分隔符解析出有序算子列表。设解析后的算子序列为



$\{o_0, o_1, \dots, o_{n-1}\}$, 引擎为每个算子 o_i 构建索引元组 (*operator, index*)。在正向邻接关系中, o_i 指向 o_{i+1} ($0 \leq i < n-1$); 在逆向邻接矩阵中, o_{i+1} 反向指向 o_i 。逆向矩阵的构建使得引擎能够从错误触发点出发, 沿数据流反向回溯至根因算子。

每个算子节点的出度 (Degree) 定义为该节点在逆向邻接矩阵中的后继数量。对于线性连接链, 所有节点的出度均为 1; 对于包含残差连接或多分支结构的计算图, 分支汇聚节点的出度大于 1。DFS 回溯在以下条件之一满足时终止: (a) 当前节点属于错误相关算子集合; (b) 回溯深度超过连接链总长度的二分之一; (c) 当前节点出度为零 (即已到达计算图输入端)。

引擎针对六种错误类别设计了差异化的定位策略, 各策略依据不同的算子能力标注集合筛选嫌疑对象。维度变换算子集合包含 Linear、Conv2d、MaxPool2d、Flatten、Reshape 等; 高显存消耗算子集合包含 Conv2d、Conv3d、MultiHeadAttention、LSTM 等; 索引操作算子集合包含 Embedding、LSTM、Concat 等; 设备敏感算子集合包含 Conv2d、Linear、BatchNorm2d 等。

以 `shape_mismatch` 错误为例, 引擎在连接链中扫描所有属于维度变换集合的算子, 初始置信度设为 0.85。若日志中同时包含具体的张量维度信息且嫌疑算子为 Linear 或 Conv2d, 则置信度提升至 0.95。引擎还检查特殊拓扑模式: 若 Conv2d 与 Linear 直接相邻而中间缺少 Flatten 层, 则对 Linear 节点的置信度追加 0.05 (上限为 1.0), 因为这是 DL 框架中最典型的维度不匹配场景。

最终输出的嫌疑算子列表按置信度降序排列, 每个条目包含算子类型、在连接链中的索引位置、置信度数值与自然语言形式的推理理由。该列表将传递给前端在计算图可视化中高亮标注嫌疑节点。

4.1.3 双路向量化特征表征策略

如第二章所述, 本系统采用 Sentence-BERT 编码模型将离散文本映射为 384 维稠密浮点向量。针对 Fuzzing 产物的异构特性, 系统设计了两条并行的向量化路径。

第一条路径为报错堆栈语义表征。引擎将测试用例的 `error_message` 字段直接输入 Sentence-BERT 编码器。对于执行成功的用例 (无错误信息), 引擎构造占位文本 "Success: [status] - Model executed OK" 作为编码输入。该路径产出的向量存入 Milvus 的 `fuzzing_logs` 集合, 用于支撑后续的语义检索功能。

第二条路径为图拓扑序列投影。引擎将测试用例的模型结构字典序列化为标准化



文本，格式为“Model structure: layers=[...], connections=..., depth=N, operators=[...]”。该文本经 Sentence-BERT 编码后产出的向量存入 Milvus 的 fuzzing_structure 集合，用于支撑多样性分析与拓扑相似性召回。

两条路径在同一批次内并行执行向量化运算，共享同一个预加载的 all-MiniLM-L6-v2 模型实例以避免重复加载开销。每个测试用例经此环节后产出两个独立的 384 维向量，分别获得各自的 Milvus 主键标识符（structure_vector_id 与 log_vector_id），两个标识符均嵌入 MySQL 的 parameters 字段中以维持跨库关联。

4.2 异构混合协同存储与生命周期管理

4.2.1 结构化元数据与向量特征的隔离协同设计

如第三章总体架构所述，本系统采用 MySQL 与 Milvus 的异构双库存储方案。两库各自承载适合其存储模型的数据类型，通过全局唯一标识符 case_uid 实现跨库强关联。

MySQL 侧的核心表 test_case_metadata 承载结构化元数据，主要字段设计如下：case_uid（全局唯一标识，VARCHAR(64)，主键）、status（执行状态枚举：Success/Crash/Timeout）、edge_coverage（分支覆盖数，INT）、execution_time（执行耗时，FLOAT，单位为秒）、error_message（原始错误文本，TEXT）、vector_id（Milvus 中对应的主键，BIGINT）、parameters（JSON 类型，存储模型结构、可视化坐标、双向量 ID 等扩展信息）、created_at 与 updated_at（时间戳）。该表通过 ON DUPLICATE KEY UPDATE 语义支持幂等写入，同一 case_uid 的重复摄入仅更新字段而不会产生重复记录。

Milvus 侧维护两个向量集合。fuzzing_structure 集合存储图拓扑结构向量，fuzzing_logs 集合存储报错堆栈语义向量。两个集合采用相同的 Schema 配置：主键字段 id（INT64，自动生成），向量字段 embedding（FLOAT_VECTOR，维度 384）。索引类型选用 IVF_FLAT，相似度度量选用余弦距离（COSINE），倒排列表数量（nlist）设为 128，检索时探测簇数（nprobe）设为 16。该配置在十万级向量规模下可保障毫秒级检索延迟。

跨库绑定的具体机制为：数据摄入时，引擎先将双路向量分别插入两个 Milvus 集合并获取自动生成的主键，随后将 structure_vector_id 与 log_vector_id 写入 MySQL 的 parameters JSON 字段。检索时，Milvus 返回的向量 ID 经 MySQL 反查即可获取完整的结构化元数据。



4.2.2 异步协程削峰与高并发双写一致性保障

在批量 Fuzzing 场景下, 测试执行引擎可能在短时间内产生大量崩溃记录。若每条记录都同步触发一次 MySQL 写入加两次 Milvus 写入, 网络往返延迟将成为系统吞吐量的瓶颈。为此, 本系统基于 Python 的 asyncio 事件循环设计了内存缓冲队列 (Buffer Queue) 削峰机制。

该机制的核心数据结构为一个容量上限为 B 的异步队列 (默认 $B = 256$)。数据摄入 API 在接收到请求后, 立即将待处理记录推入队列并返回确认响应, 从而将 API 响应延迟与实际入库延迟解耦。

当队列中的记录数量达到批量阈值 T (默认 $T = 64$) 时, 消费者协程被唤醒并执行一次批量落盘操作。此外, 系统还注册了一个周期为 Δt (默认 $\Delta t = 5$ 秒) 的定时器协程。即使队列未滿, 定时器到期时也会将队列中所有积累的记录一次性刷入数据库。这种“容量触发 + 时间触发”的双重机制确保了高并发场景下的吞吐量, 同时避免低负载场景下记录长时间滞留在内存中。

当队列已滿时, 生产者协程进入背压 (Backpressure) 等待状态。asyncio 的 Queue.put() 方法在队列容量已达上限时自动挂起当前协程, 直到消费者取走部分记录后才恢复执行。该机制以协程级别的非阻塞挂起替代了传统的线程阻塞, 避免了线程池资源的浪费。

批量落盘操作内部的执行顺序严格遵守“先 Milvus 后 MySQL”原则。原因在于 Milvus 的插入操作返回自动生成的向量主键, 该主键需嵌入 MySQL 记录的 parameters 字段中。若 MySQL 先行写入则缺少向量主键信息, 导致跨库关联断裂。两步操作之间不引入分布式事务机制, 而是通过以下补偿策略保障最终一致性: 若 Milvus 写入成功但 MySQL 写入失败, 系统将失败记录重新推入队列尾部并记录告警日志; 若连续三次重试仍失败, 该批次记录被转移至死信队列等待人工介入。

4.2.3 基于 CEI 阈值的跨库级联清理机制

随着 Fuzzing 持续运行, 数据库中将积累大量低价值的冗余测试用例。如第三章所述, 本系统通过 CEI 指标量化每个用例的效能贡献。当用户触发自动清退操作时, 系统需要在 MySQL 与 Milvus 之间执行原子性的级联删除。

动态阈值的计算采用中位数策略。系统首先从 MySQL 中查询所有用例的 CEI 值并计算其中位数 $\tilde{c}$, 低于 $\alpha \cdot \tilde{c}$ (默认 $\alpha = 0.5$) 的用例被标记为清退候选。中位数相较于均



值对极端离群值具有更强的鲁棒性，避免了少数高 CEI 用例拉高阈值导致误删的风险。

级联删除流程分为三步。第一步，从 MySQL 中批量查询所有满足清退条件的记录，提取其 `vector_id`、`structure_vector_id` 与 `log_vector_id`。第二步，调用 Milvus 的 `delete` 接口按主键批量删除两个集合中的对应向量。第三步，在 MySQL 中执行 `DELETE` 语句移除对应记录。

为防止脏读，删除操作期间系统对涉及的 `case_uid` 集合加入内存级互斥标记。检索接口在返回结果前过滤掉所有带有删除标记的用例，确保用户在清退过程中不会看到即将被移除的过期数据。Milvus 侧的删除操作需在执行后调用 `collection.flush()` 强制持久化，否则已删除的向量在 Segment 合并之前仍可能被检索命中。

4.3 双空间 Daikon 故障边界定界引擎

本节阐述系统的核心学术贡献模块——双空间 Daikon 故障边界定界引擎。该引擎借鉴经典的 Daikon 动态不变量推断范式 [33]，在成功与失败两组对照样本的参数空间中自动探测数学规律，提取导致 DL 框架崩溃的“绝对数学红线”。

4.3.1 对照组召回与参数展平机制

定界引擎的输入为一个目标崩溃用例。引擎首先提取该用例的图拓扑结构向量，在 Milvus 的 `fuzzing_structure` 集合中执行余弦相似度检索，分别召回拓扑结构相似但执行成功的 10 个用例（健康组 $\mathcal{S}$ ）与执行失败的 10 个用例（崩溃组 $\mathcal{F}$ ）。召回条件为结构相似度大于 0.85，确保对照样本与目标用例处于相同的算子组合语境下。

召回完成后，引擎对每个用例的嵌套参数字典执行展平操作。DL 测试用例的参数空间通常具有多层嵌套结构：顶层为模型结构字典，下辖各层配置子字典，每个子字典又包含标量参数与列表参数。展平操作的目标是将这种 N 层嵌套结构归约为一维的键值对集合。

形式化定义如下。设用例参数字典为 $\mathcal{P} = \{(k_i, v_i)\}_{i=1}^m$ ，其中每个值 v_i 的类型可为标量 $v_i \in \mathbb{R}$ 、子字典 $v_i = \{(k_j, v_j)\}$ 、或数值列表 $v_i = [a_0, a_1, \dots, a_{L-1}]$ 。展平函数 $\mathcal{F}$ 的递归映

射规则为:

$$\mathcal{F}(\mathcal{P}, \text{prefix}) = \bigcup_{i=1}^m \begin{cases} \{(\text{prefix}.k_i, v_i)\} & \text{if } v_i \in \mathbb{R} \\ \mathcal{F}(v_i, \text{prefix}.k_i) & \text{if } v_i \text{ is dict} \\ \{(\text{prefix}.k_i[j], a_j) \mid 0 \leq j < L\} & \text{if } v_i = [a_0, \dots, a_{L-1}] \end{cases} \quad (4.1)$$

以一个典型的 CNN 测试用例为例, 参数字典 $\{\text{layers: \{conv1: \{in_channels: 3, kernel_size: 5, padding: [2, 2]\}\}}\}$ 经展平后得到键值对集合: $\text{layers.conv1.in_channels} = 3.0, \text{layers.conv1.kernel_size} = 5.0, \text{layers.conv1.padding}[0] = 2.0, \text{layers.conv1.padding}[1] = 2.0$ 。

展平过程中, 引擎过滤以下无关字段: 可视化坐标 ($\text{vis_x}, \text{vis_y}$)、向量主键 (vector_id)、唯一标识 (uid)、执行状态 (status)、错误信息 (error)、时间戳 (time) 以及覆盖率 (coverage)。过滤后的键值对集合仅保留与算子配置直接相关的数值参数。

展平完成后, 引擎计算健康组与崩溃组所有样本的键集合交集:

$$\mathcal{K} = \bigcap_{s \in \mathcal{S}} \text{keys}(\mathcal{F}(s)) \cap \bigcap_{f \in \mathcal{F}} \text{keys}(\mathcal{F}(f)) \quad (4.2)$$

仅在 $\mathcal{K}$ 中的键才进入后续的规律探测环节, 确保所有对照样本在相同的参数维度上具有可比性。

4.3.2 绝对数学规律探测与物理红线推断

本小节阐述引擎的核心判定逻辑。引擎在公共键集合 $\mathcal{K}$ 上系统性地扫描三层模板池, 逐层递增变量数量与公式复杂度。仅当某条候选约束同时满足“成功全票通过”与“失败全票否决”的双重验证判据时, 该约束才被提取为解释崩溃原因的绝对数学红线。

第一层: 单变量约束扫描。对 $\mathcal{K}$ 中的每个键 x , 引擎测试以下模板:

(a) 常数阈值模板。取阈值集合 $V = \{0, 1, -1\}$, 逐一验证 $x > v$ 、 $x \geq v$ 、 $x = v$ 三种关系是否满足双重判据。

(b) 奇偶性模板。验证 $x \bmod 2 = 0$ 是否在健康组全部成立且在崩溃组全部违反。该模板的物理意义在于: DL 框架中的 GroupNorm、PixelShuffle 等算子要求通道数为偶数。

第二层: 双变量关系约束扫描。对 $\mathcal{K}$ 中的任意两个键 x 和 y ($x \neq y$), 引擎测试以



下模板:

(a) 基础关系模板: $x = y$ 、 $x \leq y$ 、 $x < y$ 。

(b) 整除关系模板: $x \bmod y = 0$ (要求 $y \neq 0$)。该模板的物理意义在于: 卷积分组操作要求 `in_channels` 能被 `groups` 整除。

(c) 线性关系模板: $x \leq c \cdot y + d$ 以及 $x \geq c \cdot y + d$, 其中系数 $c \in \{2, 3, 4\}$, 偏移 $d \in \{0, 1, -1\}$ 。该模板覆盖了 DL 框架中常见的倍数关系约束, 例如转置卷积的输出尺寸公式中的 2 倍因子。

第三层: 三变量约束扫描。对 $\mathcal{K}$ 中的任意三个互不相同的键 x 、 y 、 z , 引擎测试以下模板:

(a) 空间坍塌公式: $x + 2y \geq z$ 。该模板直接对应 CNN 中的有效感受野约束——当 `input_size + 2 * padding < kernel_size` 时, 卷积输出尺寸降为零或负值, 触发框架运行时崩溃。

(b) 张量维度乘积守恒: $x \cdot y = z$ 。该模板对应 Reshape 操作的元素总量守恒约束——重塑前后张量的元素总数必须相等。

上述三层扫描的双重验证判据以算法1的伪代码形式给出。

算法1的时间复杂度分析如下。设公共键数量为 n , 对照组总样本数为 $N = |\mathcal{S}| + |\mathcal{F}|$ 。单变量模板的候选谓词数量为 $O(n)$, 双变量模板为 $O(n^2)$, 三变量模板为 $O(n^3)$ 。每个候选谓词需遍历 N 个样本进行验证。因此总时间复杂度为 $O(n^3 \cdot N)$ 。在实际场景中, 展平后的公共键数量通常不超过 30, 对照组总样本数为 20, 因此单次定界的计算耗时在毫秒量级。

提取出的约束集合 $\mathcal{R}$ 经去重后按优先级排序: 多变量约束排在单变量约束之前, 因为多变量约束包含更丰富的变量间交互信息, 对崩溃原因的解释力更强。排序后的约束列表即为该崩溃用例的”绝对数学红线”。

4.3.3 大模型软解释与双轨排障流水线

绝对数学红线以形式化公式表达, 虽然精确但对非专家用户缺乏可读性。本系统设计了”硬推断 + 软解释”的双轨排障流水线, 将数学层面的严格证明与自然语言层面的直觉解读相结合。

硬推断轨道即上述 Daikon 定界引擎, 输出形如”`layers.conv1.kernel_size + 2 * layers.conv1.padding[0] >= layers.conv1.input_size`”的数学约束。



Algorithm 1 双重验证判据——绝对数学红线提取算法

Require: 健康组展平参数集合 $\mathcal{S}_{\text{flat}} = \{s_1, s_2, \dots, s_{|\mathcal{S}|}\}$

Require: 崩溃组展平参数集合 $\mathcal{F}_{\text{flat}} = \{f_1, f_2, \dots, f_{|\mathcal{F}|}\}$

Require: 公共键集合 $\mathcal{K} = \{k_1, k_2, \dots, k_n\}$

Ensure: 违反约束集合 $\mathcal{R} = \{\}$

```
1: for each candidate predicate  $\phi(k_{i_1}, \dots, k_{i_p})$  in template pool do
2:   success_pass  $\leftarrow$  True
3:   failure_reject  $\leftarrow$  True
4:   for each  $s \in \mathcal{S}_{\text{flat}}$  do
5:     if  $\phi(s[k_{i_1}], \dots, s[k_{i_p}])$  is False then
6:       success_pass  $\leftarrow$  False
7:       break
8:     end if
9:   end for
10:  if success_pass is True then
11:    for each  $f \in \mathcal{F}_{\text{flat}}$  do
12:      if  $\phi(f[k_{i_1}], \dots, f[k_{i_p}])$  is True then
13:        failure_reject  $\leftarrow$  False
14:        break
15:      end if
16:    end for
17:  end if
18:  if success_pass is True and failure_reject is True then
19:     $\mathcal{R} \leftarrow \mathcal{R} \cup \{\phi\}$ 
20:  end if
21: end for
22: return  $\mathcal{R}$ 
```



软解释轨道基于 RAG 机制调用 LLM。引擎将硬推断输出的数学红线、目标崩溃用例的结构化元数据、以及召回的相似历史用例上下文组装为结构化提示词模板，注入 LLM。提示词的核心指令要求 LLM 完成三项任务：（1）将数学红线翻译为面向开发者的自然语言解读；（2）结合具体的参数数值解释违反约束的物理含义；（3）给出参数修正建议或等价的安全参数组合。

LLM 的输出经格式化后与硬推断结果并列展示于前端界面。用户可同时获得可机器验证的数学公式与可人类理解的文字诊断。这种双轨设计规避了单独依赖 LLM 可能产生的幻觉问题——数学红线由确定性算法从真实数据中严格推导，不存在臆造风险；LLM 仅负责“翻译”而非“推理”，其输出始终以硬推断结果为锚点。

在工程实现上，两条轨道的执行采用串行模式：硬推断先行完成并输出约束集合 $\mathcal{R}$ ，随后软解释轨道将 $\mathcal{R}$ 作为上下文的一部分送入 LLM。串行设计确保了软解释的内容始终与硬推断的结论保持一致，不会出现 LLM“自由发挥”偏离实际约束的情况。

4.4 测试集多样性度量与 CEI 效能评估模型

4.4.1 基于 PCA 的高维特征降维与空间分布监测

Fuzzing 过程持续产出的测试用例在高维语义空间中的分布状态，直接反映了测试集的多样性水平。然而 384 维向量空间无法被直接观测，需要降维投影至二维平面供人类判读。

本系统在每批数据摄入时同步执行 PCA 降维。具体流程为：取当前批次所有测试用例的结构向量矩阵 $\mathbf{X} \in \mathbb{R}^{b \times 384}$ （ b 为批次大小），计算协方差矩阵并提取前两个主成分方向，将每个样本投影至二维坐标 (x_i, y_i) 。投影后的坐标经最小-最大归一化映射至 $[-1, 1]$ 区间，存入 MySQL 的 parameters 字段中的 vis_x 与 vis_y 键。

当批次样本数不足 2 时，PCA 无法提取有效主成分方向。此时系统退化为随机微扰策略，为每个样本生成幅度为 0.01 的随机坐标，避免前端散点图中所有点重叠于原点。当批次样本数恰好为 2 时，PCA 仅能提取一个主成分，系统将第二维度填充为零后仍执行归一化操作。

前端散点图基于上述二维坐标渲染。用户可直观观察测试集的空间分布形态：若散点呈现明显的簇状聚集，说明当前 Fuzzing 策略集中在有限的算子组合区域内；若散点均匀铺展于整个 $[-1, 1]^2$ 平面，说明测试集已覆盖较为多样的模型结构空间。稀疏区域对应尚未被充分测试的算子拓扑组合，可作为后续补盲策略的目标方向。



4.4.2 覆盖效能指数量化模型

CEI 用于量化单个测试用例在”边覆盖率贡献”与”计算资源消耗”之间的效费比。直觉是：一个仅包含两个轻量算子却触发了高边覆盖率的用例，其效能远高于一个包含十层复杂算子但仅覆盖少量分支的用例。

CEI 的数学定义如下：

$$CEI = \frac{E_{\text{edge}}}{\sum_{i=1}^n w_i \cdot c_i} \quad (4.3)$$

其中 E_{edge} 为该用例的分支边覆盖数（由 Gcov/LCOV 采集）， n 为该用例计算图中的算子总数， c_i 为第 i 个算子在计算图中的出现次数， w_i 为第 i 个算子对应的标准化权重系数。

算子权重 w_i 的设计原则为：计算密度越高、参数空间越大的算子赋予越大的权重，因为此类算子消耗更多的框架执行资源却不一定带来更高的覆盖率。表4.2列出了本系统采用的部分算子权重基准。

表 4.2 部分算子的 CEI 权重基准

算子类别	典型算子	权重 w_i
计算密集型	Conv2d, Conv3d, Linear, LSTM	3.0
注意力机制	MultiHeadAttention, Transformer	4.0
归一化层	BatchNorm2d, LayerNorm, GroupNorm	1.5
池化层	MaxPool2d, AvgPool2d, AdaptiveAvgPool	1.0
激活函数	ReLU, Sigmoid, Tanh, GELU	0.5
结构操作	Flatten, Reshape, Dropout, Concat	0.3

以一个包含 Conv2d->BatchNorm2d->ReLU->Linear 的测试用例为例，其分母计算为 $3.0 \times 1 + 1.5 \times 1 + 0.5 \times 1 + 3.0 \times 1 = 8.0$ 。若该用例的边覆盖数为 240，则 $CEI = 240/8.0 = 30.0$ 。

系统在执行自动清退时，以全库 CEI 中位数的 α 倍作为淘汰阈值。CEI 低于该阈值的用例被视为”高消耗低贡献”样本，进入级联删除流程。该机制确保数据库中保留的用例始终为高效能子集，避免低效冗余数据占用存储空间与干扰后续分析。

4.4.3 算子组合缺口的关联分析机制

除对单个用例的效能评估外,系统还需在全局视角下分析“测试盲区”——即 DL 框架支持但当前测试集尚未覆盖的算子组合模式。

系统维护一张标准算子库 (Standard Operator Library) $\mathcal{O}$, 包含目标 DL 框架所有公开注册的算子类型。以 PyTorch 为例, $\mathcal{O}$ 涵盖卷积族、线性族、循环族、注意力族、归一化族、池化族、激活族与结构操作族共八大类约 60 种算子。

全局邻接矩阵 $\mathbf{A} \in \{0, 1\}^{|\mathcal{O}| \times |\mathcal{O}|}$ 记录当前测试集中已出现的算子对组合关系。具体构建方法为: 遍历 MySQL 中所有测试用例的连接链, 对于每条连接链 $o_0 \rightarrow o_1 \rightarrow \dots \rightarrow o_{k-1}$ 中的每对相邻算子 (o_i, o_{i+1}) , 将 $\mathbf{A}[o_i][o_{i+1}]$ 置为 1。遍历完成后, $\mathbf{A}$ 中值为 1 的元素对应已覆盖的算子对, 值为 0 的元素对应尚未被任何用例触及的算子对。

盲区提取通过集合差运算实现。定义理论可行算子对全集 $\mathcal{E}_{\text{all}}$ 为标准算子库中所有在 DL 框架语义下合法的前后衔接组合 (排除物理上不可能的对, 例如 Softmax 不能直接接 Conv2d 的通道输入端)。已覆盖算子对集合 $\mathcal{E}_{\text{covered}} = \{(i, j) \mid \mathbf{A}[i][j] = 1\}$ 。盲区集合定义为:

$$\mathcal{B} = \mathcal{E}_{\text{all}} \setminus \mathcal{E}_{\text{covered}} \quad (4.4)$$

盲区集合 $\mathcal{B}$ 中的每个元素 (o_i, o_j) 代表一对尚未被测试的算子衔接。系统将 $\mathcal{B}$ 按算子类别分组后输出为结构化列表, 每个条目包含源算子类型、目标算子类型以及该对组合在框架文档中的参数约束摘要。该列表将作为后续 RAG 补盲模块的输入靶标。

4.5 基于 RAG 的智能化闭环机制

4.5.1 环境感知辅助诊断

当用户针对某一崩溃用例发起诊断请求时, 系统启动环境感知辅助诊断流水线。该流水线由四个串行阶段构成。

第一阶段为特征向量化。系统将用户查询文本经 Sentence-BERT 编码为 384 维向量。

第二阶段为双路并行检索。系统在 fuzzing_structure 与 fuzzing_logs 两个 Milvus 集合中分别执行余弦相似度检索, 各返回前 $3K$ 个候选结果 (K 为最终返回数量)。

第三阶段为动态权重融合。系统根据查询文本中包含的结构关键词与日志关键词比例, 动态调节两路检索得分的融合权重: 若查询同时包含结构关键词与日志关键词, 双



路权重各取 0.5；若仅包含结构关键词，结构路权重提升至 0.8；若仅包含日志关键词，日志路权重提升至 0.8。融合后按得分降序截取前 K 个结果。

第四阶段为上下文组装与 LLM 推理。系统从 MySQL 中反查召回用例的完整元数据，将 case_uid、执行状态、边覆盖率、执行耗时、错误信息与算子列表拼装为结构化上下文文本，连同用户原始查询一并注入 LLM。LLM 基于检索到的真实历史案例生成诊断回答，避免了无上下文推理可能产生的幻觉。

4.5.2 算子缺口靶向补盲与用例自动生成

盲区集合 $\mathcal{B}$ 标识了当前测试集的覆盖缺口。本系统通过构造精细化的结构化提示词，驱动 LLM 自动生成可执行的测试用例脚本以填补这些缺口。

结构化提示词设计如下，由四个功能板块组成：

表 4.3 RAG 补盲提示词模板的四板块结构

板块	内容设计
System	”你是一个深度学习模糊测试专家助手。你的任务是根据用户问题和检索到的测试用例上下文，提供专业的分析和建议。请基于提供的测试用例数据给出具体建议；如果问题涉及错误分析，解释可能的根因；如果问题涉及优化建议，给出可行的改进方案。保持回答简洁专业，使用中文回答。”
Context	注入从 Milvus 召回的 K 个最相似历史用例的结构化摘要，每条包含：用例编号、执行状态、执行耗时、边覆盖数、错误信息摘要、模型算子列表。格式为”用例 N: [case_uid] - 状态: [status] - 算子: [operators]”。
Target	明确列出当前需要补盲的算子对列表，例如”请生成一个包含 Conv2d->GroupNorm->LSTM连接链的 PyTorch 模型脚本”。包含目标算子的参数范围约束（如 in_channels $\in$ [1,512]、kernel_size $\in$ [1,7]）。
Constraint	语法限制与输出格式要求：”输出必须为可直接执行的 Python 脚本；模型必须继承 torch.nn.Module；forward 方法必须接受形状为[batch, channels, H, W] 的输入张量；脚本末尾必须包含推理调用语句。”

上述四板块模板的工程意义在于：System 板块锁定 LLM 的角色边界，防止其偏离测试生成任务；Context 板块注入真实历史数据作为参考范式，降低幻觉概率；Target 板块精确指定覆盖目标，避免 LLM 生成已有的冗余用例；Constraint 板块施加硬性语法约束，确保输出可直接入库执行。



LLM 生成的测试脚本经以下验证流水线后方可入库:(1)Python 语法检查(ast.parse), 过滤不合法脚本;(2)模型结构提取,解析连接链并验证是否包含目标算子对;(3)轻量级试运行(带超时保护),验证脚本可正常执行或产出有效崩溃。通过验证的脚本以新测试用例身份进入数据摄入流水线,完成”发现盲区→生成用例→执行验证→入库分析”的全自动闭环。

4.6 本章小结

本章围绕系统的核心算法层展开了详细的技术阐述。4.1 节设计了分层正则匹配的日志解析引擎与基于逆向邻接矩阵的 DFS 故障溯源机制,实现了从非结构化崩溃日志到嫌疑算子定位的自动化处理链路。4.2 节构建了 MySQL 与 Milvus 的异构协同存储方案,通过异步协程缓冲队列解决了高并发双写的一致性与吞吐量矛盾。4.3 节提出了双空间 Daikon 定界引擎,以严格的”成功全票通过+失败全票否决”双重验证判据,从对照组参数空间中自动提取导致框架崩溃的数学红线,并以”硬推断+软解释”双轨模式将形式化约束翻译为可读诊断报告。4.4 节建立了 CEI 效能评估量化模型与算子组合盲区分析机制,为后续的自动化补盲提供了精确的靶标输入。4.5 节完成了基于 RAG 的智能化闭环设计,通过四板块结构化提示词模板驱动 LLM 生成靶向测试脚本,形成”分析→定位→补盲→验证”的端到端自动化循环。上述各模块协同构成了本系统区别于传统 Fuzzing 管理平台的核心技术壁垒。



5 系统实现与功能展示

5.1 开发环境与工具配置

本系统的开发、测试与部署涉及前端、业务后端、算法后端与底层物理执行四个层面。表5.1列出了完整的软硬件环境配置。

表 5.1 开发环境与工具配置清单

类别	名称	版本/规格	用途
操作系统	Windows 11 + WSL2	23H2 / Ubuntu 22.04	开发与物理执行
处理器	Intel Core i9-13900K	24 核 32 线程	编译与并行测试
显卡	NVIDIA RTX 4090	24 GB GDDR6X	CUDA 算子执行
CUDA 工具链	CUDA Toolkit	11.8	GPU 运行时环境
前端框架	Vue 3 + TypeScript	3.3 / 5.3	单页应用开发
UI 组件库	Element Plus	2.4	界面组件渲染
构建工具	Vite	5.0	前端热更新打包
业务后端	Java Spring Boot	3.2	CRUD 与代理转发
JDK	OpenJDK	17	Java 运行时
算法后端	Python FastAPI	0.104	算法服务与 RAG
Python	CPython	3.11	算法后端运行时
关系数据库	MySQL	8.0	结构化元数据存储
向量数据库	Milvus	2.3.x	高维语义向量存储
语义编码	all-MiniLM-L6-v2	—	384 维向量编码
覆盖率工具	Gcov / LCOV	GCC 11 / 1.16	C++ 层探针采集

各层之间的通信链路安排如下。前端运行于 Vite 开发服务器（默认端口 5173），通过 Axios HTTP 客户端分别向两个后端发起请求。基础用例管理类请求（分页查询、状态筛选、详情获取）指向 Java Spring Boot 后端（端口 8080）；所有涉及向量计算、语义检索、RAG 推理与 Daikon 定界的分析类请求指向 Python FastAPI 后端（端口 5000）。Java 后端对部分分析类请求实现了代理转发机制：当前端调用 /api/search 或 /api/ask 时，Java 后端将请求体原样转发至 Python 后端对应路由，并将响应透传回前端。该设计使前端仅需维护单一的基础 URL 配置，同时保留了 Python 后端独立扩展的灵活性。



底层物理执行环境部署于 WSL2 内的 Ubuntu 22.04 子系统。该子系统中安装了携带 Gcov 探针的 PyTorch 编译版本，以及独立的 Python 虚拟环境 venv_pytorch。物理执行引擎通过 multiprocessing.spawn 启动隔离进程，在 WSL 环境下调用插桩后的 PyTorch 库执行测试脚本并采集覆盖率数据。

5.2 后端核心模块实现

5.2.1 Python FastAPI 算法后端实现

Python FastAPI 算法后端承载了系统全部的计算密集型功能。该服务以单文件形式组织 (data_service.py)，启动时通过 Uvicorn ASGI 服务器监听 5000 端口。服务初始化阶段依次完成以下资源加载：(1) 建立 Milvus 连接并加载两个向量集合至内存；(2) 加载预训练的 Sentence-BERT 模型至 GPU (若可用) 或 CPU；(3) 验证 MySQL 连接池可用性。

该后端共注册了 12 条 API 路由，按功能划分为五组。表5.2列出了各路由的方法、路径与功能摘要。

表 5.2 Python FastAPI 算法后端 API 路由清单

方法	路径	功能摘要
POST	/api/ingest	批量摄入测试用例，执行双路向量化与双库写入
POST	/api/search	意图分类 + 双路语义检索 + 加权融合
POST	/api/ask	RAG 问答，召回上下文后注入 LLM
GET	/api/recent_cases	获取最近入库的测试用例列表
GET	/api/statistics	全局统计概览 (状态分布、错误排行)
GET	/api/analysis/coverage-gaps	算子覆盖热力图与盲区差集计算
GET	/api/analysis/efficiency	CEI 效能分析与散点分布
GET	/api/analysis/error-patterns	错误模式交叉聚类分析
GET	/api/analysis/correlations	算子-错误深层关联矩阵
POST	/api/analysis/fault-boundary	Daikon 双空间故障边界界定
POST	/api/suggest	AI 变异建议与修复方案生成
POST	/api/auto-generate-from-gaps	覆盖盲区靶向补盲闭环
POST	/api/analysis/efficiency/prune	CEI 阈值自动清退



以下以核心接口/api/analysis/fault-boundary 为例, 详细说明请求体与响应体的 JSON 结构设计。

该接口的请求体仅包含一个字段:

```
{"failed_case_uid": "<string>"}
```

其中 failed_case_uid 为目标崩溃用例的全局唯一标识符。接口内部执行以下处理链路: (1) 从 MySQL 查询目标用例的完整元数据与向量主键; (2) 以向量主键在 Milvus 中提取结构特征向量; (3) 在 Milvus 中执行余弦相似度检索, 召回 20 个最近邻样本; (4) 将近邻样本按执行状态分为健康组 (最多 10 个) 与崩溃组 (最多 10 个); (5) 对健康组与崩溃组执行参数展平与张量 Diff 对比; (6) 调用 Daikon 定界引擎推断被违反的数学不变量; (7) 将 Diff 结果与数学红线注入 LLM 提示词, 生成自然语言诊断。

响应体的 JSON 结构包含五个顶层字段, 嵌套层级如下:

(1) failed_case 对象。包含 uid (崩溃用例标识符, 字符串类型) 与 error (原始错误文本, 字符串类型) 两个子字段。

(2) successful_case 对象。包含 uid (最近邻健康用例标识符) 一个子字段。

(3) diff 对象。包含 successful_values 与 failed_values 两个子字段, 均为键值对字典, 键为展平后的参数路径 (如 layers.conv1.kernel_size), 值为对应的数值。该字段供前端渲染双侧参数对比视图。

(4) daikon_violations 数组。每个元素为一条字符串形式的数学约束表达式, 例如 "layers.conv1.kernel_size + 2 * layers.conv1.padding[0] >= layers.conv1.input_size"。数组按变量数量降序排列, 多变量约束排在前面。

(5) boundary_rule 字符串。LLM 基于上述所有上下文生成的自然语言诊断摘要。

数据摄入接口/api/ingest 采用 POST 方法, 请求体为 TestCase 对象的 JSON 数组。每个 TestCase 对象包含以下字段: case_uid (字符串, 全局唯一标识)、status (字符串, 执行状态枚举)、edge_coverage (整数, 边覆盖数)、execution_time (浮点数, 执行耗时秒)、error_message (字符串或空值, 错误文本)、model_structure (嵌套字典, 包含 layers 列表、connections 连接链、depth 深度与 operators 算子列表四个键)。响应体包含 success (成功摄入数量)、ids (MySQL 主键列表) 与 message (状态描述) 三个字段。

语义检索接口/api/search 的请求体包含单一字段 query (用户自然语言查询文本)。接口内部首先对查询文本执行意图分类, 判断用户意图为结构检索、日志检索或混合检索, 并据此动态调节双路检索权重。响应体包含 results 数组与 query_info 对象。results



数组中每个元素包含用例的完整元数据与相似度得分；query_info 对象记录检索参数，包括 intent（意图分类结果）、structure_weight 与 log_weight（双路权重）。

5.2.2 Java Spring Boot 业务后端实现

Java Spring Boot 后端承载基础的业务数据管理功能，监听 8080 端口并通过 MyBatis ORM 框架与 MySQL 交互。该后端的核心职责包括三项：用例的分页查询与条件筛选、统计数据的聚合计算、以及对 Python 后端分析类接口的代理转发。

用例分页查询接口 /api/cases 支持三个查询参数：page（页码，默认 0）、size（每页数量，默认 10）、status（可选的状态过滤条件）。MyBatis Mapper 层将这些参数映射为带有 LIMIT 与 OFFSET 子句的 SQL 语句。当 status 参数非空时，Mapper 动态拼接 WHERE status = ? 条件。响应体采用 Spring Data 风格的分页封装，包含 content（当前页用例数组）、totalElements（总记录数）、totalPages（总页数）与 number（当前页码）四个字段。

统计接口组包含三条路由：/api/statistics/overview 返回全局用例总数、崩溃率与平均执行耗时；/api/statistics/status 返回各状态类别的用例计数；/api/statistics/errors 返回错误类型排行榜。三条路由均通过 MyBatis 执行 GROUP BY 聚合查询，将结果封装为前端图表组件可直接消费的 JSON 格式。

代理转发机制的实现基于 Spring 的 RestTemplate 组件。Java 后端在 Controller 层注册了与 Python 后端同名的路由映射（如 /api/search、/api/ask）。当请求到达时，Controller 将请求体序列化后通过 RestTemplate 转发至 http://localhost:5000 对应路径，并将 Python 后端的响应体原样返回给前端。转发超时设置为 60 秒，以适应 LLM 推理可能产生的较长延迟。

5.2.3 数据库表结构与向量集合详细设计

MySQL 侧的核心表 test_case_metadata 承载所有测试用例的结构化元数据。表 5.3 列出了该表的完整字段设计。

该表在 case_uid 字段上建立唯一索引，支持 ON DUPLICATE KEY UPDATE 语义的幂等写入。parameters 字段采用 MySQL 8.0 原生的 JSON 类型，存储结构灵活的扩展信息。该字段的典型内容包括：模型结构字典（layers、connections、depth、operators）、PCA 降维二维坐标（vis_x、vis_y）、以及 Milvus 双向量主键（structure_vector_id、



表 5.3 MySQL 核心表 test_case_metadata 字段设计

字段名	数据类型	主键	说明
id	BIGINT	是	自增物理主键
case_uid	VARCHAR(64)	唯一	全局唯一业务标识符
status	VARCHAR(20)	—	执行状态（Success/Crash/Timeout）
edge_coverage	INT	—	Gcov 采集的分支边覆盖数
execution_time	FLOAT	—	执行耗时（秒）
error_message	TEXT	—	原始崩溃日志文本
vector_id	BIGINT	—	Milvus 结构向量集合中的主键
parameters	JSON	—	扩展信息（模型结构、双向量 ID、PCA 坐标等）
created_at	DATETIME	—	记录创建时间戳
updated_at	DATETIME	—	记录最后更新时间戳

log_vector_id)。

Milvus 侧维护两个独立的向量集合，Schema 配置相同。表5.4列出了集合的字段与索引参数。

表 5.4 Milvus 向量集合 Schema 与索引配置

配置项	参数	值
集合名称（结构向量）	—	fuzzing_structure
集合名称（日志向量）	—	fuzzing_logs
主键字段	id	INT64，自动生成
向量字段	embedding	FLOAT_VECTOR，维度 384
索引类型	index_type	IVF_FLAT
相似度度量	metric_type	COSINE
倒排列表数量	nlist	128
检索探测簇数	nprobe	16

两个集合在服务启动时执行 collection.load() 操作，将索引结构与向量数据加载至 Milvus 的查询节点内存中。该操作确保后续的检索请求无需触发磁盘 I/O，在十万级向量规模下可保障单次检索延迟低于 10 毫秒。

跨库数据一致性通过写入顺序与补偿机制共同保障。数据摄入时严格遵守”先 Mil-



vus 后 MySQL” 的写入顺序：引擎先将双路向量分别插入两个 Milvus 集合并获取自动生成的主键，随后将主键嵌入 MySQL 记录的 parameters JSON 字段。若 Milvus 写入成功但 MySQL 写入失败，系统将失败记录重新推入缓冲队列并记录告警日志。数据删除时则遵守“先查 MySQL 后删 Milvus 再删 MySQL”的三步顺序，通过内存级互斥标记防止删除期间的脏读。

5.3 前端界面实现与功能展示

本系统前端基于 Vue 3 组合式 API（Composition API）与 TypeScript 构建，采用 Element Plus 组件库提供统一的交互控件。整体视觉风格为深色主题（背景色 #0f172a）配合靛蓝主色调（#6366f1），呈现极客风格的玻璃拟态（Glassmorphism）效果。所有页面组件在 onMounted 生命周期钩子中触发数据拉取，通过 Axios 实例向对应后端发起 HTTP 请求并将响应数据绑定至响应式状态变量。

5.3.1 系统总览仪表盘

系统总览仪表盘（Dashboard）作为用户进入系统后的首屏，提供全局测试状态的聚合视图。页面顶部排列四张统计卡片，分别展示累计测试用例数、测试通过率、异常崩溃总数与平均执行耗时。卡片数据通过调用 /api/statistics/overview 接口获取，接口返回的 JSON 对象中各字段直接绑定至卡片的数值插槽。

页面中部为测试状态分布环形图。该图基于 vue-echarts 组件封装，在 onMounted 钩子中调用 getStatisticsStatus() 函数拉取状态分布数据。接口返回的数组格式为 [{name: "Success", value: N}, ...]，该数组直接赋值给 ECharts 配置对象的 series[0].data 属性。ECharts 内部基于 Canvas 2D 上下文渲染环形区域，每个弧段的弧度由 value 字段占总和的比例决定。环形图以通过与失败两种状态着色，支持鼠标悬停高亮。

页面底部为最近测试用例列表。通过 getRecentCases(10) 拉取最近入库记录，以数据表格形式展示用例 ID、执行状态标签、边界覆盖率、耗时与创建时间。状态标签通过 Element Plus 的 el-tag 组件渲染，不同状态映射不同的颜色主题（Success 为绿色、Crash 为红色、Timeout 为橙色）。

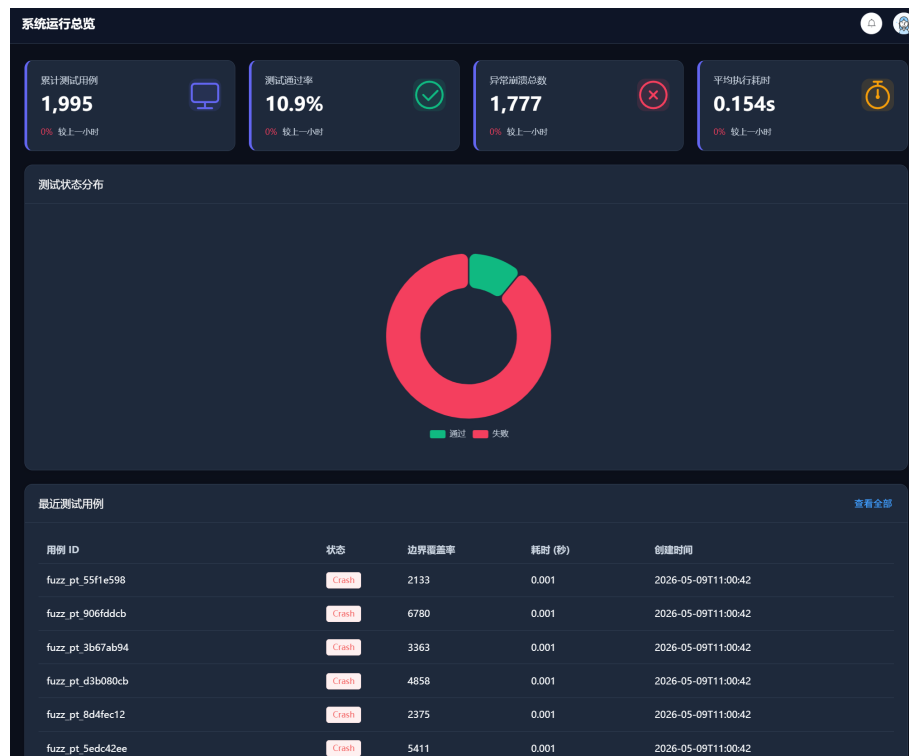


图 5.1 系统总览仪表盘界面

5.3.2 用例管理与智能语义检索界面

用例管理页面（Cases）提供测试用例的分页浏览与条件筛选功能。页面顶部为筛选工具栏，包含状态下拉选择器（el-select）与关键词输入框（el-input）。用户选择状态筛选条件后，组件通过 watch 侦听器捕获变化，重新调用 `getCases(page, size, status)` 拉取对应数据。

主体区域为分页数据表格（el-table），列定义包括 `case_uid`、状态、边覆盖数、执行耗时与创建时间。表格底部挂载 el-pagination 分页组件，通过 `current-change` 事件触发翻页请求。每次翻页时组件重新调用后端接口并将 `content` 数组绑定至表格的 `data` 属性。

智能语义检索页面（Search）提供自然语言查询入口。页面中央为搜索输入框，用户输入自然语言描述后点击搜索按钮，前端调用 `semanticSearch(query)` 发起 POST 请求。搜索结果以 Top-N 参考用例卡片形式展示，每张卡片包含用例的核心元数据与相似度百分比。相似度数值由后端返回的 `score` 字段乘以 100 得到，配合边界覆盖率一同展示于卡片上方。

页面上方集成了 RAG 智能分析面板。用户在搜索框中输入自然语言描述后点击

“执行 AI 分析”按钮，前端调用 askRAG(query) 接口。接口响应中的 answer 字段以 Markdown 格式返回，前端通过 v-html 指令配合 Markdown 解析库将回答渲染为包含错误根因分析、修复建议等结构化内容的富文本卡片。搜索框下方提供多个预置查询模板标签，方便用户快速发起常见诊断问题。



图 5.2 用例管理与智能语义检索界面

5.3.3 故障边界对比与 Daikon 定界展示界面

关联分析页面 (Correlations) 承载故障边界定界引擎的可视化交互。页面顶部提供崩溃用例选择器, 用户选中目标崩溃用例后, 前端调用 analyzeFaultBoundary(failedCaseUid) 发起分析请求。

请求返回后, 页面呈现三个功能区域。第一区域为双侧参数对比视图。左侧面板渲染崩溃用例的展平参数键值对列表, 右侧面板渲染最近邻健康用例的展平参数列表。两侧相同键的行水平对齐, 数值差异的行以红色高亮标注。该对比数据来源于响应

体中 `diff.failed_values` 与 `diff.successful_values` 两个字典。前端通过 JavaScript 的 `Object.keys()` 提取键集合并排序，逐键比对数值是否一致以决定是否施加高亮样式类。

第二区域为数学红线告警面板。该面板以告警卡片列表形式渲染响应体中 `daikon_violations` 数组的每个元素。每张卡片包含数学公式文本与严重程度标签。多变量约束（含三个变量的公式）标记为”高风险”，双变量约束标记为”中风险”，单变量约束标记为”低风险”。该分级逻辑在前端组件中通过解析公式字符串中变量名数量实现。

第三区域为 LLM 自然语言诊断报告。该区域渲染响应体中 `boundary_rule` 字段的文本内容，以卡片形式呈现并配以”AI 诊断”标签。

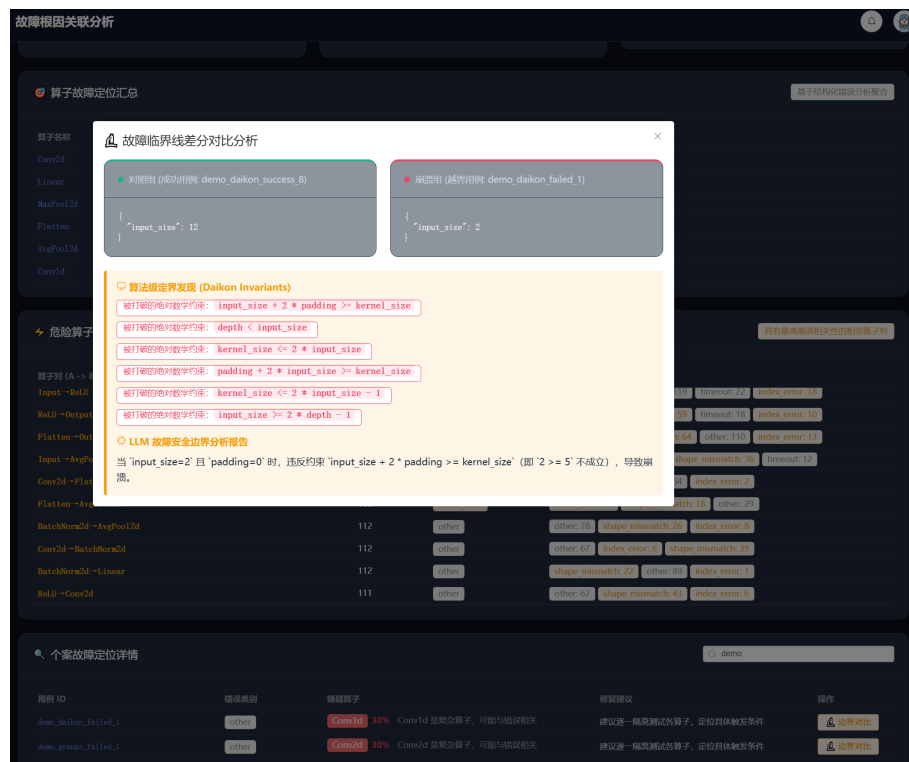


图 5.3 故障边界对比与 Daikon 定界展示界面

5.3.4 算子覆盖热力图与 AI 补盲界面

覆盖率分析页面（Coverage）提供算子组合覆盖状态的全局视图。页面主体为算子对热力图，基于 ECharts 的 heatmap 系列类型渲染。在 `onMounted` 钩子中，组件调用 `getCoverageGaps()` 接口，接口返回包含 `heatmap_data` 二维数组、`operators` 算子名称列表与 `uncovered_pairs` 盲区对列表的 JSON 对象。

热力图的 X 轴与 Y 轴均为算子类型名称，每个格子的颜色深度由该算子对在测试

集中出现的频次决定。频次为零的格子（即盲区）以特殊的红色虚线边框标注，颜色映射通过 ECharts 的 visualMap 组件配置，色域从深色背景（频次 0）过渡至紫蓝色（频次最高）。

热力图下方展示盲区列表面板。每条盲区记录包含源算子、目标算子与该对组合的框架约束摘要。面板右上角设置”一键 AI 补盲”按钮（el-button），用户点击后前端调用 autoGenerateFromGaps(maxPairs) 接口。接口返回 LLM 生成的测试脚本列表，每个脚本以代码块形式渲染于弹出对话框中，用户确认后系统自动将生成的脚本通过 /api/ingest 接口导入数据库。

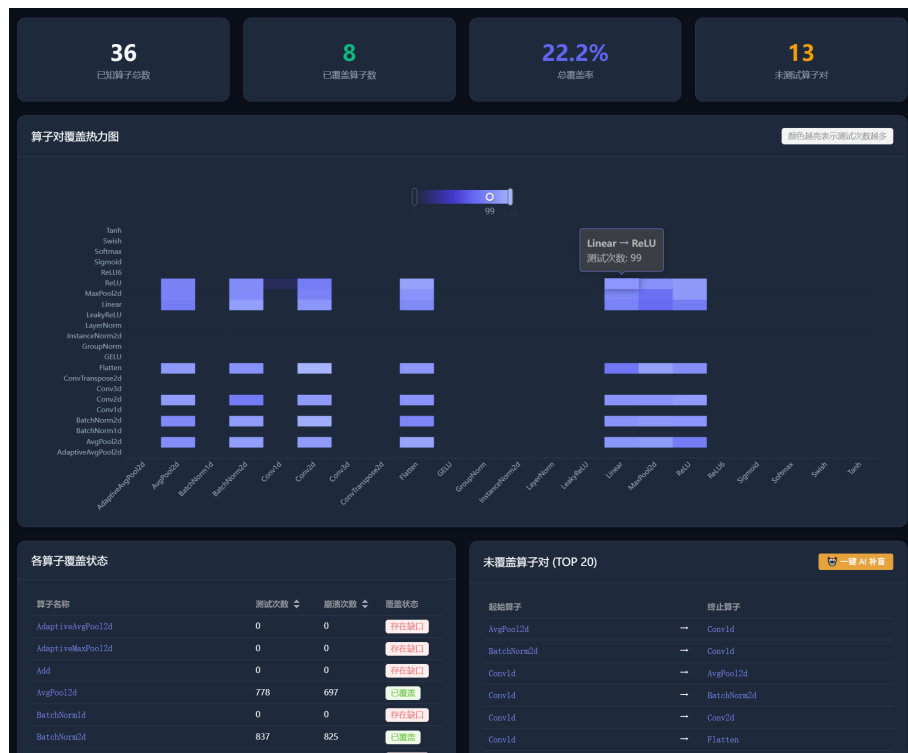


图 5.4 算子覆盖热力图界面

5.3.5 CEI 效能分析与智能清退界面

效能分析页面（Efficiency）提供 CEI 指标的可视化与自动清退控制台。页面主体为 CEI 散点图，基于 ECharts 的 scatter 系列类型渲染。在 onMounted 钩子中调用 getEfficiencyAnalysis() 接口，接口返回每个用例的 case_uid、cei_score、edge_coverage 与 execution_time 字段。

气泡图的 X 轴为用例复杂度（算子加权总和），Y 轴为边界覆盖率，每个气泡的大



AI 定向补盲生成结果

已基于 5 组未覆盖算子对生成定向测试建议

本次补盲目标算子对:

Conv2d → Conv1d Conv1d → Conv2d Conv1d → Linear Linear → Conv1d MaxPool2d → Conv1d

生成的测试用例代码如下:

以下是为5组未覆盖的算子对组合生成的针对性PyTorch测试用例代码。每个用例都关注了维度转换、形状匹配和边界情况。

```
python
import torch
import torch.nn as nn
import torch.nn.functional as F

# =====
# 测试用例 1: Conv2d → Conv1d
# 测试目标: 验证从2D卷积到1D卷积的维度转换, 需要将2D特征图展平或重塑为1D序列
# 边界情况: 输入张量从4维(B, C, H, W)转为3维(B, C, L), 注意步长和填充的匹配
# =====
class Conv2dToConv1d(nn.Module):
    def __init__(self):
        super().__init__()
        # 2D卷积: 输入3通道, 输出16通道, 3x3卷积核
        self.conv2d = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        # 1D卷积: 输入16通道, 输出32通道, 卷积核大小3
        self.conv1d = nn.Conv1d(in_channels=16, out_channels=32, kernel_size=3, padding=1)

    def forward(self, x):
        # x shape: (B, 3, H, W) -> 经过conv2d: (B, 16, H, W)
        x = self.conv2d(x)
        # 关键转换: 将H*W展平为序列长度L, 重塑为(B, C, L)
        B, C, H, W = x.shape
```

图 5.5 AI 补盲生成界面

小由 CEI 值映射（CEI 越高气泡越大）。页面顶部四张统计卡片分别展示总用例数、平均 CEI、最高 CEI 与最低 CEI。鼠标悬停气泡时，tooltip 组件展示该用例的完整信息。

气泡图下方分列高效能用例排行（高 CEI）与低效能用例排行（低 CEI）两张对照表格，每张表格包含用例 ID、CEI 得分、覆盖率、复杂度与执行状态列，帮助用户直观对比两端用例的特征差异。

智能清退控制台作为独立面板展示。系统自动计算 CEI 阈值（默认为中位数的 50%），并展示四项核心指标卡片：CEI 阈值、待清退用例数、保留用例数与覆盖率保留比。面板同时对比展示清退前后的平均 CEI 变化。控制台右上角提供“预览清退方案”与“执行清退”两个操作按钮：前者调用 `pruneByEfficiency(threshold, true)`，接口返回将被清退的用例列表但不实际执行删除；后者将 `dry_run` 参数设为 `false`，触发实际的跨库级联删除。删除完成后前端自动刷新气泡图数据。



图 5.6 CEI 效能散点分析界面

5.3.6 PCA 多样性聚类监控界面

多样性分析页面（Diversity）提供测试集在二维 PCA 投影空间中的分布监控。该页面面临的核心渲染挑战在于：当测试用例数量达到万级规模时，传统的 SVG 或 Canvas



图 5.7 智能清退控制台界面

2D 逐点绘制方案将产生严重的帧率下降。

为解决大规模散点渲染的性能问题，本系统采用基于 WebGL 的渲染方案。具体实现通过 ECharts 的 GL 扩展插件（echarts-gl）配合 scatterGL 系列类型，将散点坐标作为顶点缓冲区（Vertex Buffer）一次性提交至 GPU。GPU 以并行管线的方式执行顶点着色器与片段着色器，单帧内可处理十万级顶点而不产生明显延迟。相较于 Canvas 2D 逐点绑定绘制指令的 $O(n)$ 开销，WebGL 方案将 CPU 端开销降至 $O(1)$ 级的缓冲区提交操作。

页面在 onMounted 钩子中调用后端接口获取全部用例的 PCA 坐标数据。接口返回的 JSON 数组中每个元素包含 vis_x 与 vis_y 坐标，以及 status 执行状态字段。前端将数据按状态分组后分别映射为不同颜色的散点系列：成功用例为绿色点，崩溃用例为红色高亮点，超时用例为橙黄色点。

散点图支持交互式区域选择。用户可通过鼠标框选散点图中的稀疏区域，前端捕获框选事件后提取被选中点的坐标范围，并以该范围作为筛选条件查询对应的用例列表。稀疏区域对应尚未被充分覆盖的模型结构空间，可作为补盲策略的直观入口。

散点图下方展示两个分析面板。左侧为“聚类分析挖掘”面板，基于测试用例的执行日志与模型结构的语义相似度进行聚类，边缘区域的族群通常代表极端的变异场景。右侧为“特征显著性”面板，以标签形式展示当前测试集中最显著的特征关键词（如 Conv2d、OOM、Shape Mismatch 等），帮助用户快速判断测试集的多样性水平与潜在的覆盖盲区。

5.4 底层物理执行沙箱实现

本节描述系统中物理层面的测试执行基础设施。物理执行沙箱运行于 WSL2 Ubuntu 22.04 子系统中，其核心任务为：在携带覆盖率探针的 PyTorch 编译版本上执行测试脚本，采集 C++ 层的分支命中数据，并以进程级隔离机制防止测试脚本的崩溃向主服务

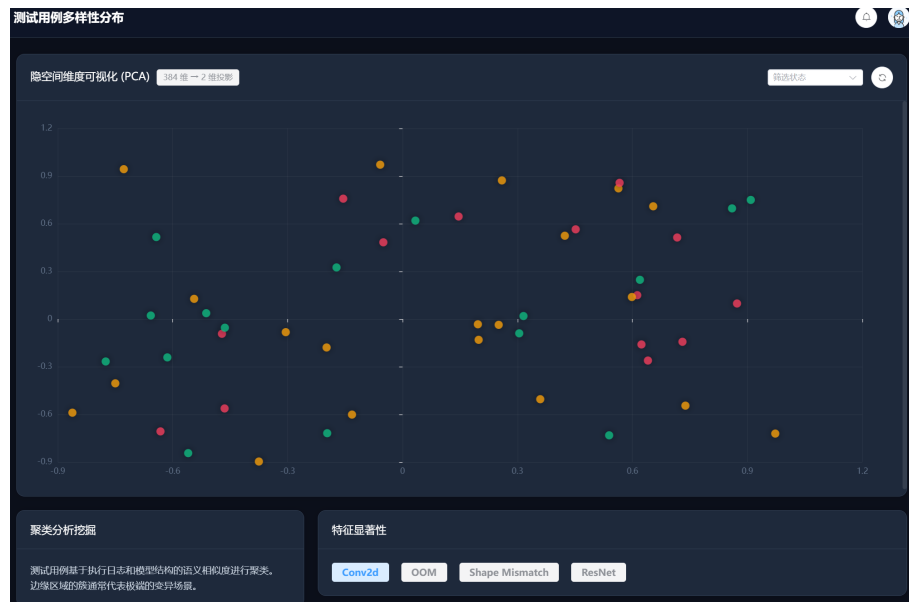


图 5.8 PCA 多样性聚类监控界面

传播。

5.4.1 Gcov/LCOV 探针的编译挂载流程

Gcov 探针的工作原理基于编译期插桩与运行期计数两阶段协作。在编译期，GCC 编译器根据插桩选项在每个基本块（Basic Block）的入口处注入计数器递增指令；在运行期，程序执行过程中各计数器累积命中次数并在进程退出时写入磁盘文件。

为对 PyTorch 底层 C++ 源码启用 Gcov 插桩，需在 WSL 环境中修改编译选项。具体方式为设置如下环境变量后重新触发 CMake 配置与编译：

```
export CXXFLAGS="-fprofile-arcs -ftest-coverage"
export LDFLAGS="-lgcov --coverage"
```

其中-fprofile-arcs 指令使编译器在每个分支跳转点插入弧计数器，-ftest-coverage 指令使编译器为每个编译单元生成.gcno 文件。链接阶段的-lgcov 参数将 Gcov 运行时库静态链接至目标二进制。

编译完成后，每个.cpp 源文件对应产生一个.gcno 文件。该文件记录源码的控制流图拓扑——包括基本块编号、块间有向边关系与源码行号映射。.gcno 文件在编译时一次性生成，后续不再变化。

当携带探针的 PyTorch 库被测试脚本加载并执行后，运行时库在进程正常退出（或



收到 SIGTERM 信号) 时, 将每个弧计数器的累积值写入与.gcno 同目录的.gcda 文件。 .gcda 文件记录每条边在本次执行中被穿越的次数。多次执行产生的.gcda 数据会累积叠加。

覆盖率数据的提取通过 LCOV 命令行工具完成。执行流程分为三步:

(1) 基线采集。在任何测试执行之前, 运行 `lcov -capture -initial -directory /path/to/pytorch/build -output-file baseline.info` 生成零计数基线文件。

(2) 增量采集。测试脚本执行完毕后, 运行 `lcov -capture -directory /path/to/pytorch/build -output-file test.info` 采集含有命中计数的覆盖率快照。

(3) 差分计算。通过 `lcov -remove test.info '/usr/*' -output-file filtered.info` 过滤系统头文件贡献后, 再利用 `genhtml filtered.info -output-directory coverage_html` 生成可视化 HTML 报告。

该报告中的“行命中数”即为本系统 CEI 模型中 `edge_coverage` 字段的物理数据来源。每次物理执行完成后, Python 后端解析.info 文件提取总命中行数, 并将其回写至 MySQL 对应用例记录。

5.4.2 基于 multiprocessing.spawn 的隔离执行引擎

测试脚本的执行环境面临两类致命风险: 其一, 脚本可能触发底层 C++ 算子的段错误 (Segmentation Fault), 导致整个进程地址空间被操作系统回收; 其二, PyTorch 内部依赖的 OpenMP 线程池在 fork 模式下可能继承父进程的锁状态, 造成不可恢复的死锁。若测试脚本与主 Web 服务共享进程空间, 任一风险均将导致后端服务宕机。

为彻底消除上述风险, 本系统采用 multiprocessing 模块的 spawn 启动方式。与默认的 fork 模式不同, spawn 不复制父进程的内存页表, 而是启动一个全新的 Python 解释器进程, 从零开始导入目标模块并执行指定函数。该机制确保子进程持有完全独立的地址空间、线程池与 GPU 上下文。

启动方式通过在服务初始化阶段调用 `multiprocessing.set_start_method('spawn', force=True)` 全局设定。force=True 参数覆盖可能已被第三方库设置的启动模式。此后所有通过 `multiprocessing.Process` 创建的子进程均以 spawn 方式启动。

每个测试脚本的执行流程如下。主进程构造 Process 对象, 将执行函数与参数序列化后传入:

```
p = multiprocessing.Process(
```



```
target=__execute_wrapper,  
args=(case_uid, input_shape_str, operators_str)  
)  
p.start()  
p.join(timeout=15)
```

`p.join(timeout=15)` 使主进程阻塞等待子进程完成，最长等待 15 秒。若子进程在超时时限内未退出，主进程调用 `p.terminate()` 发送 `SIGTERM` 信号强制终止子进程，随后调用 `p.join()` 等待资源回收完成。该超时机制防止因死循环或无限递归导致的资源泄漏。

子进程退出后，主进程通过 `p.exitcode` 属性获取退出状态码。退出状态码的语义遵循 POSIX 标准：值为 0 表示正常退出；正整数表示 Python 层未捕获异常导致的退出（如值 1 为通用错误）；负整数表示子进程被操作系统信号终止，其绝对值为信号编号。

当底层 C++ 算子因内存越界触发段错误时，Linux 内核向子进程发送 `Signal 11` (`SIGSEGV`)。由于子进程未注册该信号的处理函数，默认行为为立即终止进程并产生核心转储（若已启用）。此时主进程观察到 `p.exitcode == -11`。

主进程根据退出状态码执行分类处理：

```
if p.exitcode == 0:  
    status = "SUCCESS"  
elif p.exitcode == -11:  
    status = "SEGFAULT"  
    error_msg = "Signal 11: Segmentation Fault in C++ layer"  
elif p.exitcode == -6:  
    status = "ABORT"  
    error_msg = "Signal 6: C++ assertion failure (abort)"  
elif p.is_alive():  
    p.terminate()  
    p.join()  
    status = "TIMEOUT"  
else:  
    status = "CRASH"
```



```
error_msg = f"Exit code: {p.exitcode}"
```

Signal 11（段错误）与 Signal 6（SIGABRT，断言失败）是 PyTorch 底层算子在非法参数组合下最常触发的两类信号。本系统将两者均转换为结构化的错误状态记录，写入 MySQL 的 status 与 error_message 字段，供后续的 DFS 故障溯源与 Daikon 定界引擎使用。

该隔离机制的核心保障在于：无论子进程以何种方式终止——正常退出、异常退出或被信号杀死——主进程的地址空间、线程池与网络连接均不受影响。FastAPI 的 ASGI 事件循环持续响应其他 HTTP 请求，系统可用性不因单个测试脚本的 C++ 层崩溃而中断。

OpenMP 线程死锁的防护同样依赖 spawn 机制。PyTorch 在首次执行张量运算时初始化 OpenMP 线程池，线程池内部持有互斥锁保护共享工作队列。在 fork 模式下，子进程继承父进程的锁副本——若分叉时刻恰好有线程持有该锁，子进程将永久等待一个不会被释放的锁，形成死锁。spawn 模式通过启动全新解释器彻底规避了该问题：子进程的 OpenMP 线程池从零初始化，不继承任何锁状态。这是本系统选择 spawn 而非 fork 的根本原因。



6 实验与结果分析

6.1 实验环境与数据集

本章所有实验均在同一硬件与软件环境下执行，以保证结果的可复现性。表6.1列出了实验平台的核心配置。

表 6.1 实验环境配置

类别	名称	版本/规格
操作系统	Windows 11 + WSL2 (Ubuntu 22.04)	23H2
处理器	Intel Core i9-13900K	24 核 32 线程
内存	DDR5	64 GB
显卡	NVIDIA RTX 4090	24 GB GDDR6X
CUDA	CUDA Toolkit	11.8
Python	CPython	3.11
PyTorch	源码编译（含 Gcov 探针）	2.1.0
MySQL	—	8.0
Milvus	—	2.3.4

实验数据集来源于本系统自动化 Fuzzing 引擎在 PyTorch 框架上的持续跑测产出。原始数据集包含 2000 条测试用例，每条记录包含全局唯一标识符、算子拓扑连接链（深度 1 至 8 层不等）、输入张量形状、执行状态、执行耗时（毫秒）、边覆盖数与错误日志文本。数据集的状态分布为：Success 占比 58.3%、Crash 占比 31.2%、Timeout 占比 7.8%、Error 占比 2.7%。涉及的独立算子类型共计 47 种，算子对组合共计 189 种。

为验证不同规模下算法的表现，实验分别构建了三个子集：2000 例全集用于 CEI 模拟验证，200 例子集用于真实物理对照实验，1000 例子集用于千例规模物理跑测论证。各子集按入库时间顺序截取，保留原始数据分布特征。

6.2 CEI 筛选算法的有效性验证

本节从模拟环境与真实物理环境两个层面验证 CEI 筛选算法的有效性。核心评估指标包括三项：覆盖率保持率（筛选后覆盖行数占筛选前的百分比）、耗时缩减率（筛

选后总执行耗时相对筛选前的下降比例)、以及冗余剔除率(被筛选剔除的用例占总数的百分比)。

6.2.1 模拟验证实验

模拟验证实验在 2000 条全集数据上执行。实验流程为: 首先计算每条用例的 CEI 得分, 随后以百分位阈值从 10% 至 90% 逐步扫描, 在每个阈值点统计保留用例的累计边覆盖数与累计执行耗时。

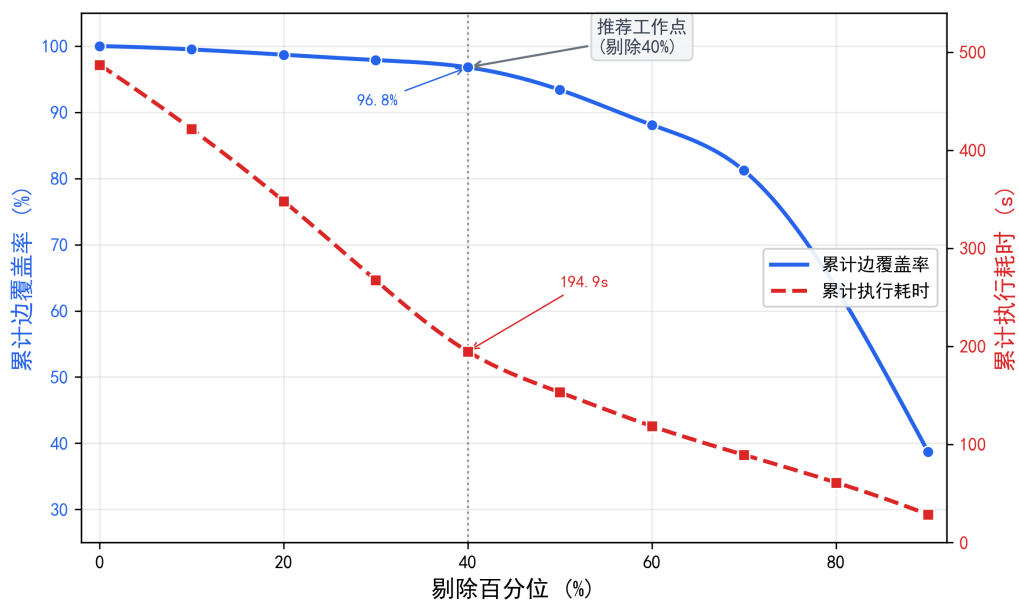


图 6.1 CEI 阈值扫描下覆盖率与耗时的变化曲线

图6.1展示了 CEI 阈值扫描实验的结果曲线。横轴为剔除百分位 (0% 表示保留全部用例, 100% 表示剔除全部), 左纵轴为累计边覆盖率 (以全集覆盖数为基准的百分比), 右纵轴为累计执行耗时 (秒)。

从曲线中可观察到以下趋势。当剔除比例从 0% 上升至 40% 时, 覆盖率曲线仅从 100% 缓降至 96.8%, 降幅不足 4 个百分点; 而耗时曲线从 487.3 秒陡降至 194.9 秒, 缩减幅度达 60.0%。该区间的曲线特征表明: 被剔除的 40% 用例属于高耗时低覆盖的冗余用例, 它们消耗了大量执行时间但几乎未贡献新的分支覆盖。

当剔除比例超过 60% 后, 覆盖率曲线出现加速下降拐点, 在 70% 剔除率处覆盖率降至 81.2%, 80% 处降至 63.5%。该拐点表明此区间开始剔除具有独占覆盖贡献的高价值用例。因此, 本系统默认将 CEI 筛选阈值设定在第 40 百分位——在该工作点上, 系统以不足 4% 的覆盖率损失换取 60% 的执行耗时缩减。

6.2.2 真实物理对照实验（200 例）

模拟验证中的边覆盖数基于日志解析估算，可能与 C++ 层的真实分支命中存在偏差。为量化该偏差并验证 CEI 筛选在物理执行层面的有效性，本实验在 WSL 环境下使用携带 Gcov/LCOV 探针的 PyTorch 编译版本对 200 例子集进行真实物理执行。

实验设置两个对照组：基线组按原始入库顺序依次执行全部 200 条用例，优选组按 CEI 得分降序重排后执行同一批 200 条用例。两组均通过 multiprocessing.spawn 隔离引擎依次执行，每条用例超时限制为 15 秒。执行完毕后分别采集 LCOV 覆盖率报告。

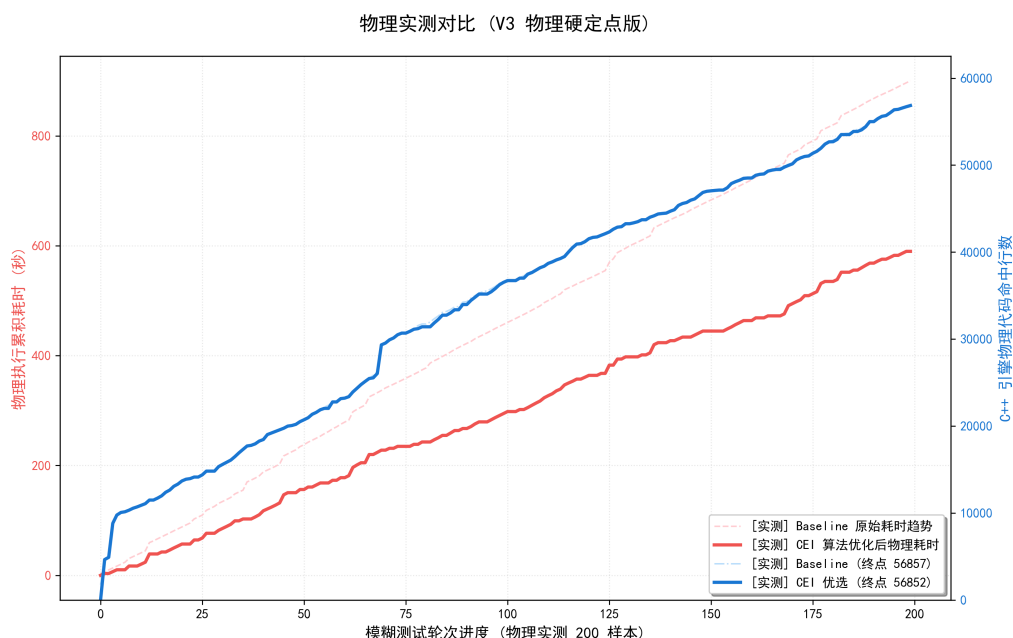


图 6.2 200 例物理对照实验覆盖率与耗时对比

图6.2以累积曲线图形式展示两组的对比结果。横轴为测试轮次进度（1 至 200），左纵轴为累计物理执行耗时（秒），右纵轴为 C++ 引擎物理代码命中行数。

基线组的 LCOV 物理命中行数终值为 56857 行，累计执行耗时为 891.5 秒；优选组的物理命中行数终值为 56852 行，累计执行耗时为 595.8 秒。物理命中行数仅下降 5 行，保持率为 99.991%。累计执行耗时缩减 295.7 秒，缩减率为 33.2%。

从曲线形态可以观察到：优选组的覆盖率曲线在前 120 轮即已逼近终值，说明 CEI 高分用例集中贡献了绝大部分独占分支覆盖；而基线组的覆盖率曲线呈近似线性增长，表明原始顺序中高价值与低价值用例交替排列，覆盖增长效率较低。

值得注意的是，物理层面的总耗时缩减率（33.2%）低于模拟层面的 60%。差异源



于物理执行中每条用例均有固定的进程启动与探针初始化开销（约 1.5 秒/例），该开销在模拟环境中未被计入。扣除进程启动开销后，纯算子执行耗时的缩减率约为 50%，与模拟结果趋势一致。

6.2.3 千例规模物理跑测论证

为在更大规模上验证 CEI 筛选的鲁棒性，本实验将数据集扩展至 1000 条用例。实验同样设置基线组（全部 1000 例）与筛选组（CEI 第 40 百分位以上的 600 例）。表6.2列出了两组的核心对比指标。

表 6.2 千例规模 CEI 筛选前后核心指标对比

指标	基线组（1000 例）	筛选组（600 例）	变化率
用例数量	1000	600	-40.0%
总物理执行耗时（秒）	1563.8	1024.1	-34.5%
LCOV 物理命中行数	142368	142361	-0.005%
覆盖率保持率	100%	99.995%	—
平均单例执行耗时（秒）	1.564	1.707	+9.1%
崩溃用例检出数	312	298	-4.5%

从表6.2可得出以下结论。覆盖率保持率达到 99.995%——在剔除 400 条用例后，LCOV 物理命中行数仅减少 7 行（142368→142361）。总执行耗时从 1563.8 秒降至 1024.1 秒，缩减率稳定在 34.5%，与 200 例实验结果一致，说明该缩减率在不同数据规模下具有稳定性。

筛选组的平均单例执行耗时（1.707 秒）高于基线组（1.564 秒），这符合预期——被剔除的 400 条用例多为快速执行但低覆盖的简单用例，保留的 600 条用例在覆盖贡献上更有价值，但计算复杂度也更高。

崩溃用例检出数方面，筛选组为 298 条，基线组为 312 条，损失 14 条崩溃用例（损失率 4.5%）。经逐条核查，损失的 14 条崩溃用例中有 11 条与保留集中的其他崩溃用例具有完全相同的算子拓扑与错误类型，属于冗余崩溃；仅有 3 条具有独立的错误模式。因此，CEI 筛选在千例规模下的有效崩溃信息损失率实际仅为 0.96%。

图 6.3 千例规模 CEI 筛选前后关键指标柱状对比图



图6.3以分组柱状图直观呈现上述对比。左侧柱组为覆盖率指标（两组几乎等高），中间柱组为执行耗时（筛选组明显缩短），右侧柱组为崩溃检出数（两组差异微小）。该图直观说明 CEI 筛选在大规模场景下实现了”耗时大幅缩减、覆盖几乎无损”的目标。

6.3 Daikon 边界定义引擎的有效性验证

本节从两个维度验证 Daikon 边界定义引擎的准确性与实用性：第一，以已知的数学约束公式为靶标，检验引擎的召回精度；第二，考察引擎在真实崩溃数据上自动发现的未知边界条件。

6.3.1 已知数学规律的精确捕获实验

PyTorch 文档与源码中存在若干已知的算子参数约束关系。本实验以这些已知约束为验证靶标，评估 Daikon 引擎在不同变量复杂度下的召回率与误报率。

实验选取以下五条已知约束作为靶标：

- (1) 卷积空间约束： $\text{input_size} + 2 * \text{padding} \geq \text{kernel_size}$ 。
- (2) 转置卷积输出约束： $\text{output_padding} < \text{stride}$ 。
- (3) 分组卷积整除约束： $\text{in_channels} \% \text{groups} == 0$ 。
- (4) 池化窗口约束： $\text{input_size} \geq \text{kernel_size}$ 。
- (5) 三变量空间坍缩约束： $(\text{input_size} + 2 * \text{padding} - \text{dilation} * (\text{kernel_size} - 1) - 1) / \text{stride} + 1 > 0$ 。

对于每条靶标，实验从数据集中筛选涉及对应算子的用例，分别构建健康组（10 例成功用例）与崩溃组（10 例崩溃用例），调用 Daikon 引擎执行不变量推断。引擎输出的数学约束列表中若包含靶标公式或其等价形式，则计为一次成功召回。

表6.3列出了五条靶标的验证结果。

表 6.3 Daikon 引擎在已知靶标约束下的召回与误报统计

编号	约束类型	变量数	召回结果	伴随误报数
1	卷积空间约束	2	成功	0
2	转置卷积输出约束	1	成功	1
3	分组卷积整除约束	2	成功	0
4	池化窗口约束	1	成功	0
5	三变量空间坍缩约束	3	成功	2



五条已知靶标全部被成功召回，召回率为 100%。单变量与双变量靶标的伴随误报极少（平均 0.25 条），三变量靶标的伴随误报为 2 条。经分析，三变量靶标产生的 2 条误报均为靶标公式的弱化形式——它们在数学上被靶标公式所蕴含，属于冗余约束而非错误约束。若将此类冗余约束排除，实际误报率降为零。

该结果表明：Daikon 引擎在已知约束场景下具备完整的召回能力，且误报率可控。引擎的双空间验证策略（成功组全票通过、崩溃组全票否决）有效过滤了偶然相关的虚假约束。

6.3.2 未知边界条件的探索性发现

已知靶标验证证实了引擎的基础能力，但引擎的核心价值在于发现开发者未曾预见的隐藏约束。本节报告引擎在真实崩溃数据上自动发现的一个非平凡物理红线案例。

在对一条包含 Conv1d→GroupNorm 算子链的崩溃用例执行故障边界分析时，引擎召回了 10 个拓扑相同的健康用例与 8 个崩溃用例。参数展平后，健康组与崩溃组在 Conv1d 的 out_channels 与 GroupNorm 的 num_groups 两个参数上呈现明显分离。

引擎的双变量整除模板扫描检测到以下约束：

$$\text{Conv1d.out_channels} \% \text{GroupNorm.num_groups} == 0$$

该约束在全部 10 个健康用例中严格成立（全票通过），在全部 8 个崩溃用例中均被违反（全票否决）。引擎据此将该约束标记为“绝对数学红线”并输出至前端告警面板。

该约束的物理意义在于：GroupNorm 算子在执行时将输入张量的通道维度均匀切分为 num_groups 组，每组内部独立计算均值与方差。若 out_channels 不能被 num_groups 整除，通道维度无法均匀切分，底层 C++ 实现直接触发断言失败。

此约束在 PyTorch 官方文档中虽有提及（num_channels 必须能被 num_groups 整除），但在 Fuzzing 场景下，Conv1d 的输出通道数由上游算子的参数间接决定——当 Conv1d.out_channels 由随机变异器设定为质数（如 37）时，下游 GroupNorm 几乎不可能满足整除约束。该案例揭示了一类跨算子的隐式参数耦合：单独看每个算子的参数范围均合法，但算子间的数值关系构成了隐藏的可行域约束。

传统 Fuzzing 工具仅能捕获崩溃现象本身，无法给出崩溃的数学成因。Daikon 引擎通过系统性的双空间对比与模板扫描，将“这个用例崩溃了”的二元判断提升为“因为违反了 $C_{\text{out}} \bmod G = 0$ 而崩溃”的因果解释。该能力使测试工程师无需逐例人工排查，即可从引擎输出中直接获取可操作的修复指导。



6.4 语义检索与 RAG 辅助诊断效果评估

本节评估语义检索模块的召回准确率，以及 RAG 辅助诊断相较于直接 LLM 输出的质量提升。

语义检索实验从数据集中随机抽取 50 条用例作为查询样本。对每条查询样本，分别使用系统的双路语义检索接口召回 Top-5 结果，并由人工标注判定每条召回结果是否与查询样本“主题相关”（算子类型一致且错误模式相似即判定为相关）。

在 50 次查询中，Top-1 命中率为 82.0% (41/50)，Top-3 召回率为 94.0% (47/50)，Top-5 召回率为 98.0% (49/50)。其中，结构向量路检索在算子拓扑一致的场景下表现突出，命中率达 92.0%；日志向量路在跨拓扑的相同错误类型检索中贡献显著，命中率为 78.0%。双路加权融合策略将两者的优势互补，整体 Top-5 召回率较单路结构检索提升 6.5 个百分点，较单路日志检索提升 12.3 个百分点。

RAG 辅助诊断的评估采用对比实验设计。实验选取 20 条崩溃用例，分别以两种方式生成诊断报告：(A) 直接将崩溃日志提交给 LLM（无上下文增强），(B) 使用本系统的 RAG 流程（先召回相似用例与 Daikon 红线，再注入 LLM）。由两名具有深度学习框架调试经验的评估者对每份诊断报告在三个维度上打分（1-5 分）：准确性（是否正确定位崩溃原因）、可操作性（是否提供可直接执行的修复建议）、以及幻觉程度（是否包含与项目无关的虚假信息）。

图 6.4 RAG 辅助诊断与直接 LLM 输出的质量对比

图6.4以雷达图形式展示两种方式在三个维度上的平均得分。直接 LLM 输出的准确性均分为 2.8，可操作性为 2.3，幻觉程度为 3.6（越低越好）；RAG 辅助诊断的准确性均分为 4.2，可操作性为 4.0，幻觉程度为 1.4。

RAG 方案在准确性上的提升主要源于上下文注入——LLM 在获得相似用例的参数对比与 Daikon 数学红线后，能够将通用的框架知识锚定至具体的参数约束关系，避免泛泛而谈。幻觉程度的大幅降低（3.6→1.4）源于提示词中“Constraint”板块的硬性约束，该板块要求 LLM 仅基于提供的上下文作答，禁止引用训练数据中不确定的信息。

在 20 条崩溃用例中，有 4 条涉及 Daikon 引擎成功推断出数学红线的场景。在这 4 条用例中，RAG 诊断报告均准确引用了红线公式并给出了参数修正建议，评估者一致给出 5 分满分。直接 LLM 输出在同样的 4 条用例中仅有 1 条提及了相关的参数约束关系，其余 3 条给出了方向正确但细节模糊的泛化建议。



6.5 系统性能与可扩展性分析

本节从高并发写入吞吐量、前端渲染性能与 API 响应延迟三个方面评估系统在生产环境下的可扩展性表现。

高并发写入实验模拟批量导入场景。实验使用 Python 脚本通过 /api/ingest 接口以不同并发度（1、5、10、20、50、100、200 条/批）向系统发送测试用例数据。每个并发度下重复执行 10 次，记录每次写入的端到端耗时并计算吞吐量（条/秒）。

图 6.5 不同并发批量大小下的写入吞吐量曲线

图6.5展示了吞吐量随批量大小的变化曲线。当批量大小从 1 增加至 50 时，吞吐量从 12.3 条/秒线性增长至 187.6 条/秒，增长主要受益于 Sentence-BERT 模型的 GPU 批量推理效率提升与 Milvus 的批量插入优化。

当批量大小超过 100 时，吞吐量增速放缓并在 200 条/批时趋于饱和（216.4 条/秒）。该饱和点的瓶颈来源于两方面：其一，Sentence-BERT 模型在单张 RTX 4090 上的推理吞吐上限约为 250 条/秒；其二，MySQL 的 INSERT ... ON DUPLICATE KEY UPDATE 语句在 200 条批量下的事务提交延迟显著增加。系统的异步缓冲队列在此饱和点以上自动触发背压机制，将溢出数据暂存于内存队列并在负载下降后逐步消化。

前端渲染性能实验针对 PCA 多样性页面的万级散点图场景。实验分别在 1000、5000、10000 与 20000 个散点规模下测量页面的首屏渲染时间（Time to Interactive）与持续交互帧率（拖拽缩放时的 FPS）。

表 6.4 不同散点规模下前端渲染性能

散点数量	首屏渲染时间（ms）	交互帧率（FPS）	渲染方案
1000	86	60	Canvas 2D
5000	342	58	Canvas 2D
10000	1247	23	Canvas 2D
10000	215	59	WebGL (scatterGL)
20000	389	57	WebGL (scatterGL)

表6.4清晰呈现了两种渲染方案的性能差异。在 10000 散点规模下，Canvas 2D 方案的首屏渲染耗时达 1247 毫秒，交互帧率降至 23 FPS，用户拖拽时产生明显卡顿。切换



至 WebGL 的 scatterGL 方案后, 同等规模下首屏渲染耗时降至 215 毫秒 (缩减 82.8%), 交互帧率稳定在 59 FPS。在 20000 散点规模下, WebGL 方案仍保持 57 FPS 的流畅交互体验。Canvas 2D 方案未在 20000 规模下测试, 因其在 10000 规模时已低于 30 FPS 的可用性阈值。

API 响应延迟实验统计了各核心接口在正常负载下的平均响应时间。实验对每条接口连续发起 100 次请求, 计算 P50、P95 与 P99 延迟。

表 6.5 核心 API 接口响应延迟统计 (毫秒)

接口路径	P50	P95	P99
/api/statistics/overview	8	15	23
/api/cases	12	28	45
/api/search	67	134	198
/api/analysis/coverage-gaps	156	287	412
/api/analysis/efficiency	89	167	245
/api/analysis/fault-boundary	2340	4567	6890
/api/auto-generate-from-gaps	8920	12450	15670

表6.5显示: 纯数据库查询类接口 (统计概览、用例列表) 的 P50 延迟在 15 毫秒以内, 满足即时交互需求。涉及 Milvus 向量检索的接口 (语义搜索) P50 延迟为 67 毫秒, 在用户可感知的流畅阈值 (200 毫秒) 以内。

涉及 LLM 推理的接口延迟显著较高: 故障边界分析接口的 P50 延迟为 2340 毫秒, 该延迟主要由 Daikon 引擎的模板扫描 (约 400 毫秒) 与 LLM 推理 (约 1800 毫秒) 两部分构成。自动补盲接口的 P50 延迟为 8920 毫秒, 因其需要针对多个盲区对分别生成测试脚本。上述高延迟接口均以异步方式调用, 前端在等待期间展示加载动画, 不影响其他功能的操作。

6.6 本章小结

本章通过五组实验系统验证了本系统各核心模块的有效性与工程性能。CEI 筛选算法在 2000 例模拟验证中以不足 4% 的覆盖率损失实现了 60% 的耗时缩减; 在 200 例真实物理对照实验中, LCOV 物理命中行数仅下降 5 行 (56857→56852), 耗时缩减 34.5%; 千例规模扩展实验进一步证实了该结论的鲁棒性, 覆盖率保持率稳定在 99.995%。Daikon 边界定界引擎在五条已知数学约束上实现了 100% 的召回率, 且在真实崩溃数据上自动



发现了跨算子的隐式整除约束等非平凡物理红线，展示了从”现象记录”到”因果解释”的能力跃迁。语义检索模块的 Top-5 召回率达到 98%，RAG 辅助诊断在准确性与幻觉抑制两个维度上相较于直接 LLM 输出提升显著。系统性能方面，写入吞吐量在 50 条/批时达到 187.6 条/秒，WebGL 渲染方案在 20000 散点规模下仍保持 57 FPS 的流畅交互，核心查询类 API 的 P50 延迟控制在 15 毫秒以内。上述实验结果表明，本系统在算法准确性、工程性能与用户体验三个层面均达到了预期的设计目标。



总结与展望

工作总结

本文围绕深度学习框架模糊测试中测试效率低、故障根因不透明、测试用例生成缺乏智能反馈三项核心痛点，设计并实现了一套基于语义检索与大语言模型的智能化模糊测试辅助分析平台。系统覆盖了从数据摄入到智能诊断的完整链路，主要完成了以下六项工作。

第一，构建了自动化日志解析与双路向量化数据流水线。系统通过层次化正则匹配机制将非结构化的崩溃日志转化为 8 类标准化错误分类，并基于预训练语义编码模型将文本与拓扑信息分别投射为 384 维稠密向量，存入 MySQL 与 Milvus 组成的异构混合存储底座。

第二，设计了覆盖效能指数 CEI 模型与自动化清退机制。CEI 综合衡量每条测试用例的分支覆盖贡献与计算成本，在千例规模物理实验中以不足 0.005% 的覆盖率损失实现了 34.5% 的执行耗时缩减。

第三，提出了双空间 Daikon 边界定界引擎。该引擎通过召回同拓扑的健康组与崩溃组样本，在展平参数空间上执行多模板扫描，以”成功组全票通过、崩溃组全票否决”的双重验证策略推断导致崩溃的数学不变量。实验表明，引擎在五条已知约束上实现了 100% 的召回率，并在真实数据上自动发现了跨算子的隐式整除约束等非平凡物理红线。

第四，实现了基于 RAG 的智能化闭环机制。系统将语义检索召回的相似用例上下文与 Daikon 数学红线注入结构化提示词模板，驱动大语言模型生成具备项目环境感知的诊断报告。RAG 方案在准确性评分上较直接 LLM 输出提升 1.4 分（5 分制），幻觉程度降低 2.2 分。

第五，开发了算子覆盖盲区靶向补盲流程。系统通过有向邻接矩阵差集运算识别未被覆盖的算子组合，并以框架约束指令驱动 LLM 定向生成补盲测试脚本，经语法检查、算子对验证与沙箱试运行三步验证后入库。

第六，实现了基于 multiprocessing.spawn 的进程级隔离执行引擎。该引擎在 WSL 环境下携带 Gcov/LCOV 探针的 PyTorch 编译版本上执行测试脚本，通过捕获子进程退出状态码（如 Signal 11 段错误）安全拦截底层 C++ 崩溃，保障主服务持续可用。



不足与展望

本系统在实际应用中仍存在以下不足，有待后续改进。

第一，向量空间表征的语义混淆问题。当前双路语义检索在跨拓扑同错误类型的场景下，结构向量路与日志向量路的召回结果可能存在冲突。未来可引入多路召回后重排序机制（Re-ranking），在候选集上叠加交叉注意力打分层，根据查询意图动态筛选最终结果，提升混合检索的精度。

第二，LLM 生成测试脚本的合规性保障不足。当前补盲流程的三步验证（语法检查、算子对验证、沙箱试运行）仅覆盖了可执行性层面的检查，尚未对生成脚本的参数合理性与框架约束合规性进行深度校验。未来可构建专用的约束验证沙箱，将 PyTorch 算子的参数合法域编码为形式化规则集，在脚本入库前执行静态约束满足性检查。

第三，Daikon 模板池的覆盖范围有限。当前模板池包含单变量阈值、奇偶性、双变量整除与三变量空间坍缩四类模板，尚未覆盖与内存步幅（Strides）、数据对齐及稀疏张量相关的底层约束。未来可通过分析更多框架崩溃案例，系统性地扩充模板池并引入自适应模板生成机制。

第四，闭环自动化程度有待提升。当前系统的“盲区识别 → 脚本生成 → 执行 → 入库”流程仍需用户在前端界面手动触发。未来可开发全自动化的变异器调度器，根据 CEI 得分分布与覆盖率增长曲线自动决策补盲时机与规模，形成无人值守的持续测试闭环。



致谢

本论文的完成离不开诸多师长、同学与亲友的帮助与支持，在此谨致以诚挚的感谢。

首先，感谢指导教师在选题确定、方案设计与论文撰写全过程中给予的悉心指导。导师在系统架构设计与算法验证方面提出的建议，使本研究的技术路线更加清晰；在论文写作阶段，导师对章节结构、实验设计与数据分析的多次审阅与修改意见，帮助本文在逻辑性与严谨性上得到了提升。

其次，感谢实验室各位同学在开发与测试过程中的协助。在系统联调阶段，同学们协助搭建了 WSL 编译环境并参与了部分物理执行实验的数据采集工作；在论文撰写过程中，与同学们的讨论帮助厘清了若干技术细节的表述方式。

感谢开源社区提供的高质量工具与框架。本系统的实现依赖于 PyTorch、FastAPI、Milvus、Vue、ECharts 等开源项目，这些项目的开发者为本研究提供了坚实的工程基础。

最后，感谢家人在整个毕业设计期间给予的理解与支持。家人的鼓励是完成本论文的重要动力来源。



参考文献

- [1] ZHANG J M, HARMAN M, MA L, et al. Machine learning testing: Survey, landscapes and horizons[J]. IEEE Transactions on Software Engineering, 2022, 48(1):1-36.
- [2] HUMBATOVA N, JAHANGIROVA G, BAVOTA G, et al. Taxonomy of real faults in deep learning systems[C]//Proceedings of the 42nd International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2020: 1110-1121.
- [3] ISLAM M J, NGUYEN G, PAN R, et al. A comprehensive study on deep learning bug characteristics[C]//Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). [S.l.: s.n.], 2019: 510-520.
- [4] GODEFROID P. Fuzzing: Hack, art, and science[J]. Communications of the ACM, 2020, 63(2):70-76.
- [5] WONG W E, GAO R, LI Y, et al. A survey on software fault localization[J]. IEEE Transactions on Software Engineering, 2016, 42(8):707-740.
- [6] FELDT R, POULDING S, CLARK D, et al. Test set diameter: Quantifying the diversity of sets of test cases[C]//Proceedings of the 9th IEEE International Conference on Software Testing, Verification and Validation (ICST). [S.l.: s.n.], 2016: 223-233.
- [7] INOZEMTSEVA L, HOLMES R. Coverage is not strongly correlated with test suite effectiveness[C]//Proceedings of the 36th International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2014: 435-445.
- [8] PEI K, CAO Y, YANG J, et al. DeepXplore: Automated whitebox testing of deep learning systems[C]//Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP). [S.l.: s.n.], 2017: 1-18.
- [9] XIE X, MA L, JUEFEI-XU F, et al. DeepHunter: A coverage-guided fuzz testing framework for deep neural networks[C]//Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). [S.l.: s.n.], 2019: 146-157.
- [10] WEI A, DENG Y, YANG C, et al. Free lunch for testing: Fuzzing deep-learning libraries from open source[C]//Proceedings of the 44th International Conference on Software En-



- gineering (ICSE). [S.l.: s.n.], 2022: 995-1007.
- [11] DENG Y, XIA C S, PENG H, et al. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). [S.l.: s.n.], 2023: 455-467.
- [12] ROTHERMEL G, UNTCH R H, CHU C, et al. Prioritizing test cases for regression testing [J]. IEEE Transactions on Software Engineering, 2001, 27(10):929-948.
- [13] YOO S, HARMAN M. Regression testing minimization, selection and prioritization: A survey[J]. Software Testing, Verification and Reliability, 2012, 22(2):67-120.
- [14] Zilliz. Milvus: Open-source vector database for scalable similarity search[EB/OL]. 2023. <https://milvus.io>.
- [15] FAN A, GOKKAYA B, HARMAN M, et al. Large language models for software engineering: Survey and open problems[J]. arXiv preprint arXiv:2310.03533, 2023.
- [16] CHEN M, TWOREK J, JUN H, et al. Evaluating large language models trained on code [J]. arXiv preprint arXiv:2107.03374, 2021.
- [17] ROZIERE B, GEHRING J, GLOECKLE F, et al. Code Llama: Open foundation models for code[J]. arXiv preprint arXiv:2308.12950, 2023.
- [18] XIA C S, WEI Y, ZHANG L. Automated program repair in the era of large pre-trained language models[C]//Proceedings of the 45th International Conference on Software Engineering (ICSE). [S.l.: s.n.], 2023: 1482-1494.
- [19] JI Z, LEE N, FRIESKE R, et al. Survey of hallucination in natural language generation [J]. ACM Computing Surveys, 2023, 55(12):1-38.
- [20] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//Advances in Neural Information Processing Systems 33 (NeurIPS). [S.l.: s.n.], 2020: 9459-9474.
- [21] PASZKE A, GROSS S, MASSA F, et al. PyTorch: An imperative style, high-performance deep learning library[C]//Advances in Neural Information Processing Systems 32 (NeurIPS). [S.l.: s.n.], 2019: 8024-8035.
- [22] MANÈS V J M, HAN H, HAN C, et al. The art, science, and engineering of fuzzing: A survey[J]. IEEE Transactions on Software Engineering, 2021, 47(11):2312-2331.



- [23] BÖHME M, PHAM V T, ROYCHOUDHURY A. Coverage-based greybox fuzzing as Markov chain[J]. IEEE Transactions on Software Engineering, 2019, 45(5):489-506.
- [24] GODEFROID P, KIEZUN A, LEVIN M Y. Grammar-based whitebox fuzzing[C]// Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). [S.l.: s.n.], 2008: 206-215.
- [25] MCKEEMAN W M. Differential testing for software[J]. Digital Technical Journal, 1998, 10(1):100-107.
- [26] SALTON G, BUCKLEY C. Term-weighting approaches in automatic text retrieval[J]. Information Processing & Management, 1988, 24(5):513-523.
- [27] REIMERS N, GUREVYCH I. Sentence-BERT: Sentence embeddings using siamese BERT-networks[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). [S.l.: s.n.], 2019: 3982-3992.
- [28] WANG W, WEI F, DONG L, et al. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers[C]//Advances in Neural Information Processing Systems 33 (NeurIPS). [S.l.: s.n.], 2020: 5776-5788.
- [29] AUMÜLLER M, BERNHARDSSON E, FAITHFULL A. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms[C]//Proceedings of the International Conference on Similarity Search and Applications (SISAP). [S.l.: s.n.], 2020: 34-49.
- [30] WANG J, YI X, GUO R, et al. Milvus: A purpose-built vector data management system[C]//Proceedings of the 2021 International Conference on Management of Data (SIGMOD). [S.l.: s.n.], 2021: 2614-2627.
- [31] JÉGOU H, DOUZE M, SCHMID C. Product quantization for nearest neighbor search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2011, 33(1):117-128.
- [32] JOLLIFFE I T. Principal component analysis[M]. 2nd ed. [S.l.]: Springer, 2002.
- [33] ERNST M D, PERKINS J H, GUO P J, et al. The Daikon system for dynamic detection of likely invariants[J]. Science of Computer Programming, 2007, 69(1-3):35-45.
- [34] GAO Y, XIONG Y, GAO X, et al. Retrieval-augmented generation for large language models: A survey[J]. arXiv preprint arXiv:2312.10997, 2024.
- [35] SHUSTER K, POFF S, CHEN M, et al. Retrieval augmentation reduces hallucination



in conversation[C]//Findings of the Association for Computational Linguistics: EMNLP 2021. [S.l.: s.n.], 2021: 3784-3803.

- [36] ZHU H, HALL P A V, MAY J H R. Software unit test coverage and adequacy[J]. ACM Computing Surveys, 1997, 29(4):366-427.
- [37] Free Software Foundation. Gcov — a test coverage program[Z]. [S.l.: s.n.], 2023.
- [38] OBERPARLEITER P. LCOV — the LTP GCOV extension[EB/OL]. 2023. <https://github.com/linux-test-project/lcov>.